

CHAPTER I: INTRODUCTION

1.1 Background

Efficient money management is crucial in today's fast-paced, digital world. However, financial literacy remains low in developing countries like Nepal. A study by Nepal Rastra Bank (2022) reports a national financial literacy rate of only 57.9%, with notable disparities: males outperform females by 7.5%, and only 27.9% of those aged 60+ are financially literate compared to 63.2% of individuals aged 18–30. These gaps contribute to poor budgeting, limited savings, and financial insecurity.

WealthWise aims to address this issue through a tech-driven personal finance management platform. The global FinTech sector, which currently represents just 2% of financial services revenue, is projected to grow to \$1.5 trillion by 2030—driven largely by the Asia-Pacific region (Goyal et al., 2023). In Nepal, despite growing mobile banking and digital wallet usage, financial inclusion remains uneven, creating demand for accessible, localized financial tools.

WealthWise leverages AI, data visualization, and mobile technology to promote financial literacy and better money management. Inspired by the 50/30/20 budgeting rule (Warren & Tyagi, 2005), it helps users track spending, identify waste, and improve savings habits.

This initiative was driven by personal experiences observing poor money management among peers, particularly teens. WealthWise seeks to bridge the literacy gap by offering practical, user-friendly solutions tailored to the financial needs of individuals in Nepal.

1.2 Problem Statement

The core issue WealthWise aims to solve is the widespread lack of financial literacy and effective budgeting tools, particularly in regions like Nepal. With only 57.9% of the population financially literate (2022, Nepal Rastra Bank), many individuals struggle to allocate income wisely, often leading to overspending, insufficient savings, and extreme debt. Existing financial management systems are generally subscription-based, complex, or culturally misaligned, resulting in underserved populations.

Almost 25% of the individuals in Nepal live below the poverty line, face financial stress, missing savings opportunities, and increased vulnerability to economic shocks (World Bank, 2023). Poor financial planning can lead to bankruptcy or restrict growth for small enterprises with limited cash flow. So, Technology has the potential to break the cycle of poverty and uncertainty by improving economic stability for youths, adults, families and communities.

1.3 Objectives

This project aims to address the key challenges faced by people to manage their finance by setting up clear and focused objectives. Specifically, the objectives of this project are:

- i. To develop a finance management web application using the 50/30/20 budgeting method.
- ii. To provide data visualizations using Chart.js, that tracks their financial health with automated email notifications and chatbot support.

1.4 Literature Review

The literature review provides a comprehensive overview of existing research and systems relevant to the project, drawing insights from various sources such as academic papers, articles, and operational applications.

1.4.1 Overview of Existing System

The landscape of financial management applications is rich with tools designed to assist users in budgeting, tracking expenses, and achieving financial goals, particularly for individuals. Recent research highlights several free-to-use systems that align closely with the objectives of WealthWise, a financial management web application focused on the 50/30/20 budgeting method. Below is a detailed overview of these systems and their key features, emphasizing their relevance to personal and small business financial management in contexts like Nepal, where financial literacy is a challenge.

Goodbudget: Developed by Dayspring Technologies, Goodbudget is a budgeting app that employs the envelope budgeting system, where users allocate funds to specific categories or “envelopes” to manage spending. It is free for basic use, supporting one account and two devices, and relies on manual entry, although it is not made for Asian countries and asks for International payment options. Goodbudget allows users to share budgets, facilitating collaboration for families. Its simplicity and focus on disciplined spending make it a strong comparator to WealthWise, though it lacks AI-driven features (Dayspring Technologies, n.d.).

Rocket Money: Previously known as Truebill, Rocket Money is a personal finance app that excels in subscription management, bill negotiation, and budgeting. It offers free basic features, including budget creation and spending tracking, and is designed to help users save money by identifying and canceling unnecessary subscriptions. Rocket Money also provides credit score monitoring and net worth tracking in its Premium plan. Its proactive approach to cost-saving aligns with WealthWise’s goal of reducing unnecessary spending, though it lacks a specific budgeting methodology like 50/30/20 and also requires to compulsorily link credit cards

or International Bank account not suitable for countries like Nepal (Rocket Money, Inc., n.d.).

EveryDollar: Based on Dave Ramsey’s budgeting principles, EveryDollar allows users to create and manage budgets manually, making it accessible globally. The free version supports basic budgeting, focusing on assigning every dollar a purpose, similar to zero-based budgeting. Its straightforward approach is ideal for beginners, including teenagers, but it lacks advanced features like AI-driven advice (Ramsey Solutions, n.d.).

PocketSmith: PocketSmith offers a free basic plan that includes manual transaction importing, cash flow projections, and budget scenarios. It is particularly useful for users who want to forecast their financial future, making it relevant for small businesses aiming to grow profits. Its manual entry support ensures usability in regions with limited banking integration (PocketSmith Limited, n.d.).

YNAB (You Need A Budget): YNAB (You Need A Budget) is a powerful budgeting application that prioritizes zero-based budgeting, assigning each dollar to a specific purpose and promoting goal-oriented financial planning. The free trial offers features such as real-time transaction syncing, budget building, and goal monitoring. YNAB's teaching approach helps consumers prioritize savings and debt repayment. However, the complete version requires a membership and relies on international payment methods and bank syncing along with all transactions recorded in US Dollars, which may limit accessibility in Nepal compared to WealthWise's free web-based platform which is based on Nepalese Currency (YNAB LLC, n.d.).

These systems collectively address budgeting, expense tracking, and financial planning, but they often lack the specific 50/30/20 budgeting focus, AI-driven advice, and Nepal-specific design that WealthWise offers, making it a unique solution for enhancing financial literacy and management.

1.4.2 Technologies Used in Similar Systems

Table 1.1: Technological Comparison of Financial Management Systems

System	Programming	Languages	Databases	Frameworks
Goodbudget		Python, JavaScript	Likely PostgreSQL	Django, React
Rocket Money		Python, JavaScript	PostgreSQL, MongoDB	React, Node.js
EveryDollar		Ruby, JavaScript	SQLite	Ruby on Rails

PocketSmith	Python, JavaScript	MongoDB	React, Node.js
YNAB	Ruby, JavaScript	PostgreSQL	Ruby on Rails, React
WealthWise	Python, JavaScript	MySQL	Flask, Chart.js, Bootstrap

1.4.3 Theoretical Background

WealthWise is based on fundamental computer ideas necessary for modern financial management applications:

- a) Web Development: WealthWise's backend uses Flask, which allows seamless user interactions.
- b) Databases: MySQL maintains user data, transactions, and budgets to ensure efficient data processing.
- c) Machine Learning: OllamaLLM powers the AI chatbot, using natural language.
- d) Data Visualization: Chart.js, Matplotlib, Seaborn provides interactive financial charts to improve user understanding.
- e) Security: Security measures include 2FA Authentication which includes an OTP based login, Bcrypt to protect sensitive data in line with industry standards.

These concepts ensure WealthWise is functional, secure, and user-centric, aligning with trends in similar systems.

1.4.4 Comparative Study

Each of the budgeting tools offered by Goodbudget, Rocket Money, EveryDollar, PocketSmith, and YNAB is inferior than WealthWise for Nepali consumers. Envelope budgeting as implemented by Goodbudget would not be useful in Nepal since it does not have the AI functionality such as WealthWise with a chatbot and must international being found successful in aiding youths, the purchase means of the premium access are payment methods. small businesses are thrifty in their expenditures. Despite the fact that Rocket Money is great at dealing with subscriptions, paying and negotiating bills, its reliance on credit cards or overseas the existence of accounts in banks makes it not possible to Nepalese customers, who feel more comfortable working with WealthWise manual entry. AI-driven advice, and data analytics at WealthWise WealthWise AI provides investment and asset allocation advice and data visualization to maximize the performance of wealth. enthusiasm is lacking in the course of zero-based budgeting made simple within EveryDollar. It is suitable to teenagers and operates on manual entry in Nepal. Although PocketSmith has

the capability of manually entering and providing strong cash flow predictions to small companies. it does not have the same level of AI-powered real-time coaching of WealthWise to customers with little money literacy. The free access and manual entry WealthWise are detailed. The target audience is Nepalese customers, whereas YNAB is a membership company and multinational. self-imposed payment requirements hinder its vigorous zero-based budgeting and tracking of goals features.

1.4.5 Research Papers and Related Works

WealthWise is a financial management solution designed specifically for students, teenagers and adults. Its design is based on a wealth of scholarly research on data visualization, FinTech innovation, financial literacy, and financial inclusion tactics.

Chy and Buadi (2023) emphasize how data visualization helps non-experts understand financial information. In order to address Nepal's poor financial literacy rating of 57.9%, WealthWise takes advantage of this understanding by presenting user-friendly expenditure charts using Chart.js (Nepal Rastra Bank, 2022)

Global financial inclusion is driven by FinTech services and stakeholder involvement, according to Ha et al. (2025). This validates WealthWise's business model as a free, regional substitute for premium systems like YNAB, which are frequently unavailable in low-income environments.

On the same note, Amnas et al. (2024) observe the importance of digital financial literacy to FinTech adoption, which justified the integration of the AI chatbot of WealthWise to provide personal services guidance.

Biswakarma et al. (2025) mention the place of user innovativeness in the Nepalese FinTech industry where the uptake, which is an area that WealthWise tackles using interactive apps such as 50/30/20 budgeting rule.

Frydman, and Camerer (2016) investigate biases in the decisions about financial choices which is countered by WealthWise with systematic assistance.

Perdana et al. (2018) demonstrate how interactive visual tools, a crucial part of WealthWise's design, help non-professional users make wise judgments.

According to Abad-Segura et al. (2020), AI and web technologies are the key to FinTech development, as it is also implemented by WealthWise with the capabilities of Flask, MySQL, and AI-based features. Singh (2023) also emphasises the role of AI as a way of minimizing financial errors strengthening the capacity of the platform which is also used by WealthWise.

According to Garg and Singh (2018) the financial awareness among the youth is low worldwide so this educational method provided by WealthWise is justified. The

50/30/20 rule identified by Warren and Tyagi (2005) makes budgeting less complicated and helps to maintain financial stability.

According to Investopedia (n.d.), financial management implies optimal use of resources which is Wealthwise's core objective.

Despite obstacles including Nepal's low rate of digital adoption (Smith, 2020; Biswakarma et al., 2025), WealthWise ensures usability by providing free access and manual entry. Multilingual support and mobile integration are planned future additions.

In conclusion, WealthWise provides a comprehensive, technologically advanced financial management.

1.5 Development Methodology

WealthWise has a structured research and development of the system procedure to make the system fit the user requirements under the premises of a solo project. This part describes the data collection, development process, and methodology methods employed.

1.5.1 Research and Development Process

WealthWise was established with a thorough awareness of the financial management challenges faced by Nepali youth and everyday customers. The first study looked at developments in financial literacy, including Nepal Rastra Bank's (2022) findings of 57.9% literacy rate, in addition to evaluating current systems like Goodbudget and YNAB to identify their strengths, weaknesses, and areas for improvement. The focus was on creating an easy-to-use, Nepal-specific tool that uses the 50/30/20 budgeting technique to encourage thrift and controlled spending. Technologies suitable for a single project were also examined during the study phase, including Chart.js for visualizations, MySQL for data storage, and Flask for backend development. The project proceeded to the development phase after gathering sufficient information and resources, where it concentrated on creating an intuitive user experience and creating fundamental features including tracking transactions, creating a budget, and utilizing OllamaLLM to integrate an AI chatbot for financial advice.

1.5.2 Project Framework

This project was developed using the Spiral Model, which emphasizes gradual feature addition, risk analysis, and iterative cycles all of which are perfect for a single project with shifting requirements. In contrast to the rigid and sequential Waterfall Model, the Spiral Model permitted testing and improvement following the development of each module, enabling the inclusion of new features based on user feedback and self-evaluation. To guarantee a reliable and functional system, the project was divided into four-phase iterative cycles.

1) Planning

During this phase, the project's objectives, target audience in Nepal, and major obstacles, like a lack of financial literacy, were defined. Technology choices like Flask, MySQL, Chart.js, OllamaLLM and timelines were also planned.

2) Risk Analysis

Potential risks, such as limited resources for a solo project and the complexity of integrating various features, were identified. Mitigation strategies included using lightweight frameworks like Flask and leveraging open-source tools like Chart.js.

3) Development and Testing

Each cycle focused on developing a specific module (e.g., user authentication, transaction tracking, chatbot integration), followed by immediate testing to ensure functionality. For example, after implementing secure authentication with Bcrypt, login functionality was tested before proceeding to the next module.

4) Evaluation and Feature Addition

After each cycle, the developed module was evaluated through self-testing and informal feedback from peers. New features were added based on insights gained, such as enhancing the dashboard with Chart.js visualizations after completing the budgeting module.

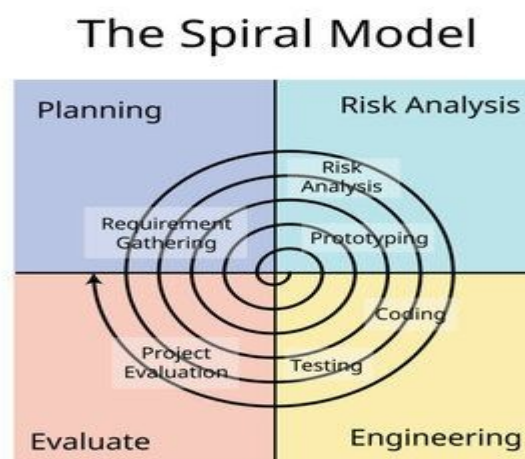


Figure 1.1: Spiral Model as Project Framework (Piscine26, n.d.)

1.5.3 Data Collection Methods

To validate WealthWise's applicability and develop its features, a mix of primary and secondary data collection approaches were used. Initially, personal observations of financial patterns among peers and local small enterprises in Nepal led

the project's course, emphasizing the importance of a basic budgeting tool. As the project report writing process began, a literature analysis was undertaken utilizing academic sources where Ha et al. (2025) and Biswakarma et al. (2025) focused on literature works to better understand FinTech trends, financial literacy gaps, and user needs in Nepal, proving the need for a free Nepal-specific financial tool. A comparative analysis of systems such as YNAB, Goodbudget, and Rocket Money was also conducted to evaluate their benefits and weaknesses, resulting in WealthWise's distinct characteristics, such as its Nepal-specific design and free access. Informal feedback was obtained by sharing the project concept with a few friends and teachers who praised the simplicity of the 50/30/20 budgeting system as well as the AI chatbot's ability to provide financial advice. These methods, while informal, guaranteed that WealthWise's development was consistent.

1.6 Scope and Limitations

A project's scope sets its boundaries and outlines the components and outcomes it will provide. It outlines the characteristics, capabilities, and goals that the project seeks to accomplish.

Limitations are the restrictions or limitations that keep the project from accomplishing its larger objectives or adding specific features. These could result from financial, time, or technological limitations.

a) Scope

WealthWise is an online financial management tool for teenagers in Nepal that emphasizes the 50/30/20 budgeting approach to encourage stability and financial awareness. The following functionalities are included in the system:

- i. **User Authentication:** Secure login via OTP and password hashing using Bcrypt to protect user data.
- ii. **Transaction Tracking:** Manual entry of income and expenses, categorized into needs, wants, and savings based on the 50/30/20 rule.
- iii. **Budget Planning:** Automated allocation of income into 50% needs, 30% wants, and 20% savings, with real-time tracking of budget adherence.
- iv. **Data Visualization:** Interactive charts powered by Chart.js to display spending patterns and budget status, aiding user understanding.
- v. **AI Chatbot:** An OllamaLLM-powered chatbot providing personalized financial advice and answering user queries to enhance financial literacy.
- vi. **Automated Notifications:** Email alerts via Mailpit for budget and financial activities, ensuring users stay informed.
- vii. **Offline Reports:** Export of transaction data as Excel files and charts as PNG images for offline analysis or sharing.

b) Limitations

Certain features are not included in WealthWise due to time and resource constraints:

- i. **Mobile Application:** WealthWise is web-based and does not currently offer a dedicated mobile app.
- ii. **Bank Integration:** The system relies on manual entry and does not support automatic bank syncing, unlike tools like YNAB or Rocket Money.
- iii. **Multi-Language Support:** The application is in English only, excluding Nepali language support, which could enhance accessibility.
- iv. **Advanced Analytics:** Features like predictive forecasting or detailed trend analysis (e.g., offered by PocketSmith) are not included.

1.7 Report Organization

Table 1.2: Report Structure Overview

Chapter No.	Chapter Title	Description
Chapter 1	Introduction	-Provides an overview of the project, objectives, problem statement, scope, and significance. -Discusses existing research, technologies and related works that support the project.
Chapter 2	System Development Process	
	System Analysis	Explains user requirements, functional and nonfunctional requirements, and feasibility analysis.
	System Design	Describes system architecture, data flow diagrams (DFD), entity-relationship diagrams (ERD), and interface design.
	Implementation	Covers coding standards, naming conventions, frameworks, and database design used in development.
	Testing & Evaluation	Describes testing strategies, test cases, and system evaluation results.
Chapter 3	Conclusion & Recommendations	Summarizes findings, achievements, challenges, and future enhancements.
	References	Lists books, research papers, websites and other sources cited in the project.
	Appendices	Includes screenshots, and additional information.

CHAPTER II: SYSTEM DEVELOPMENT PROCESS

2.1 Analysis

Identification, analysis, validation, and management of requirements are the goals of the analysis phase of a system development process. The analysis establishes the framework for creating a strong system that is suited to its users and divides needs into functional and non-functional categories. This stage of the procedure entails a comprehensive assessment to make sure the system satisfies the requirements of all parties involved, including individual users.

2.1.1 Requirement Analysis

The process of determining, evaluating, verifying, and overseeing the software or system requirements is known as requirement analysis. Through casual conversations and peer surveys, this technique takes stakeholders' and individual users' needs into account. Simplifying financial tracking while maintaining accessibility, security, and usability is the aim. Requirements can be categorized into: functional and non-functional requirements.

2.1.1.1 Functional Requirement

Functional requirements define the specific features and capabilities that WealthWise, a web-based financial management system, must have to meet the demands of its target users, Nepalese teenagers. These requirements define the system's behavior and functionality, ensuring that it meets the problem of overspending and a lack of financial goals, as observed personally among friends. The goal of identifying these objectives is to give a clear roadmap for development, direct the implementation process, and ensure that the system provides realistic solutions for financial literacy and stability using the 50/30/20 budgeting method.

a) List of Functional Requirements

The following are the expected functionalities of WealthWise, designed to simplify financial management and promote disciplined spending:

Table 2.1: Functional Requirements Table

Requirement ID	Requirement Description	Inputs	Outputs
FR-1	User should be able to register an account.	Name, Email, Mobile Number, Address, Password, Account Type	Confirmation email sent
FR-2	User should be able to log in to the system.	Email or Mobile Number, Password	Dashboard page loaded

FR-03	User should be able to track income and expenses manually.	Amount, Category (Needs/Wants/Savings), Date	Updated transaction list on dashboard
FR-04	System should automate budget planning using 50/30/20 rule.	Income amount	Displayed budget allocation (50/30/20)
FR-05	User should be able to view real-time data visualizations.	Transaction data	Interactive charts (e.g., pie chart)
FR-06	System should provide AI-powered financial advice via chatbot.	User query (e.g., "How to save?")	Personalized advice (e.g., "Reduce wants")
FR-07	System should send automated email notifications.	Budget threshold data	Email alert (e.g., "80% of wants budget used")
FR-08	User should be able to generate offline reports.	Transaction data, chart data	Excel file, PNG image of charts

b) Use Case Scenario

Users can interact with the system in the form of following use cases:

Table 2.2: User Registration

Use-Case Identifier	UC1: User Registration
Primary Actor	New User
Secondary Actor	Database
Steps	1. Navigate to the registration page 2. Enter name, mobile number, address email, password 3. Confirm password 4. Click "Register" 5. System verifies details 6. If valid and confirmed, system sends an email 7. Registration Successful
Pre-condition	User details must be unique (email/phone not already registered).
Post-condition	The user account is created, and a welcome email is sent.

Table 2.3: User Login

Use-Case Identifier	UC2: User Login
Primary Actor	Registered User
Secondary Actor	Database
Steps	1) Navigate to login page 2) Enter registered email or phone number 3) Enter Password 4) If valid, user is navigated to dashboard page 5) System sends email about login 6) Login Successful
Pre-condition	The user must have valid and verified credentials.
Post-condition	The user gains access to the dashboard and can manage finances.

Table 2.4: Transaction Management

Use-Case Identifier	UC3: Manage Transactions
Primary Actor	Logged-In User
Secondary Actor	Database
Steps	1) Navigate to Transactions section from the dashboard 2) Click on Add Income or Add Expense 3) Fill in valid details 4) Click on Add button 5) Transactions added successful
Pre-condition	The user must be logged in with valid credentials.
Post-condition	Transactions are recorded, and the dashboard updates accordingly.

Table 2.5: Budget Allocation

Use-Case Identifier	UC4: Allocate Budget
Primary Actor	System
Secondary Actor	User, Database
Description	The system automatically allocates income using the 50/30/20 rule.
Pre-condition	The user must have income transactions recorded.

Post-condition	Budget percentages are updated in the database and displayed on the dashboard.
----------------	--

Table 2.6: Financial Advice via Chatbot

Use-Case Identifier	UC5: Access Financial Advice
Primary Actor	Logged-In User
Secondary Actor	OllamaLLM API, Database
Steps	1) From the dashboard page navigate to chatbot page 2) Type the queries 3) Click on send
Pre-condition	The user must be logged in.
Post-condition	The chatbot provides relevant financial advice based on database records of the user.

Table 2.7: Report Generation

Use-Case Identifier	UC6: Generate Financial Reports
Primary Actor	User
Secondary Actor	Database
Steps	1) From the dashboard page click on Generate Reports 2) Check the files on offline folder
Pre-condition	The user must be logged in and have transactions recorded.
Post-condition	Files are saved in the Offline reports folder, and a success message is shown.

Table 2.8: User Registration

Use-Case Identifier	UC7: User Logout
Primary Actor	Logged-In User
Secondary Actor	Database
Steps	1) Click on logout button 2) System ends the session 3) System redirects to login page automatically
Pre-condition	User details must be unique (email/phone not already registered).
Post-condition	The user account is created, and a welcome email is sent.

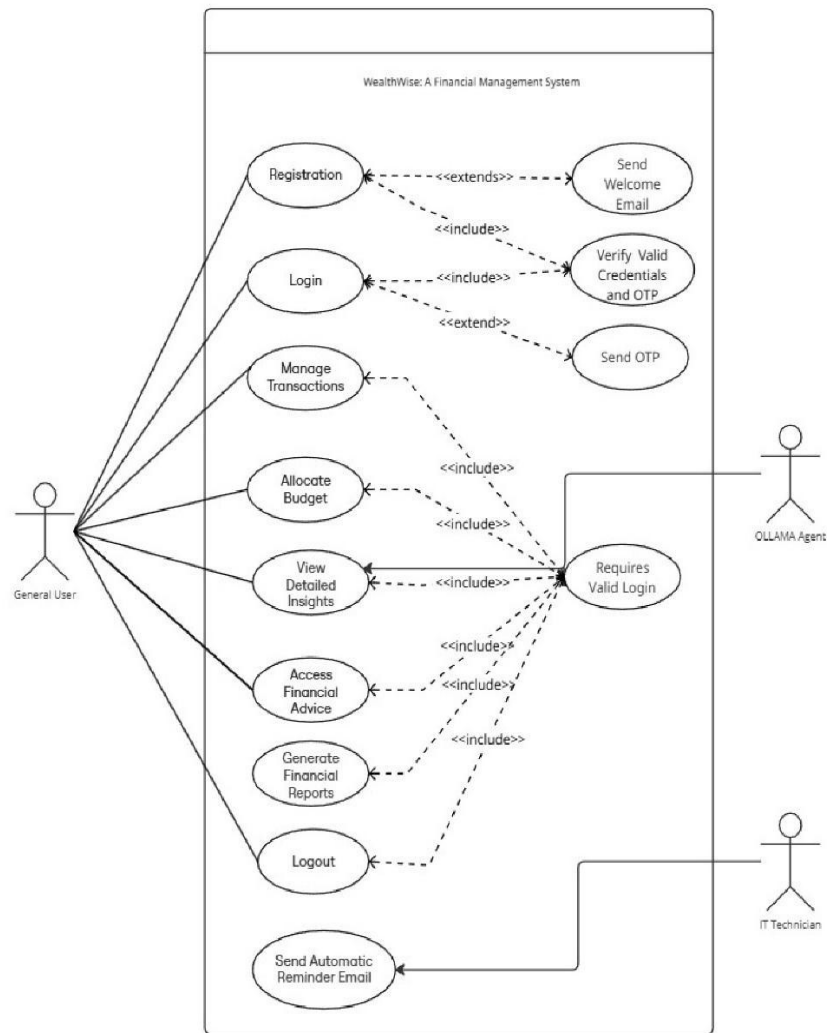


Figure 2.1: Use Case Diagram

c) User Roles & Permissions:

WealthWise defines the following user roles with corresponding access levels:

Table 2.9: User Roles and Permissions

User Role	Description	Permissions
General User	Regular user managing their personal finances	Register, log in/out, add/edit/delete income, expenses and savings, view dashboards, generate reports, use AI chatbot, receive emails
IT Technician/Support	Developer/authorized support for maintenance	System maintenance, user data troubleshooting, schedule automated reminder emails

AI Financial Advisor	Integrated chatbot for financial guidance	Provides advice and answers to user queries.
----------------------	---	--

d) System Inputs and Outputs:

The system requires specific inputs to function and produces corresponding outputs to support user activities:

Table 2.10: System Inputs and Outputs

Input	Description	Expected Output
User registration	Name, email, phone number, address, password entered by new user	Success message, Confirmation email sent to user's email address
User login	Email, password, OTP entered for authentication	Dashboard page loaded with userspecific financial data
Transaction tracking	Income/expense amount, category (needs, wants, savings), date	Transaction recorded; budget and charts updated; budget allocation (e.g., 50/30/20) displayed
Chatbot query	User's financial question (e.g., "How to save money?")	Personalized financial advice shown (e.g., "Reduce wants spending by 10%")
Budget planning	User income and expense data	Automated budget allocation breakdown (e.g., 50/30/20 rule) displayed
Data visualization	User's transaction history	Interactive charts and graphs showing spending patterns
Notifications	Budget thresholds reached or reminders scheduled	Email alerts sent for budget warnings or scheduled reminders
Offline reports	User requests data export	Downloadable Excel file with transaction data and PNG image(s) of generated charts

2.1.1.2 Non-Functional Requirements:

Non-functional requirements (NFRs) define the quality attributes, performance constraints, and operational characteristics of WealthWise. Unlike functional requirements, which specify "what" the system does (e.g., automate budgeting), NFRs describe "how well" the system performs in terms of speed, security, usability, and scalability. These requirements ensure that WealthWise operates efficiently, securely, and user-friendly for a diverse audience in Nepal.

The following table outlines the non-functional requirements for WealthWise, categorized by performance, security, usability, and scalability:

Table 2.11: Non-Functional Requirements Table

Requirement ID	Category	Description	Example
NFR-1	Performance	The system should load the dashboard within 5 seconds for 90% of users.	The webpage should not take more than 3 seconds to load after login.
NFR-2	Performance	The system should process budget calculations (50/30/20 rule) within 5 second.	Budget allocation for RS 25,000 should be displayed in under 5 second.
NFR3	Security	Users' passwords must be hashed and encrypted before storing.	Implement Bcrypt for password hashing and AES encryption for data security.
NFR-4	Usability	The interface should be intuitive, requiring no more than 4 clicks to access key features (e.g., transaction tracking, charts).	Users can navigate from dashboard to transaction entry in 3 clicks or fewer.
NFR-5	Usability	The system should provide clear instructions for the AI chatbot to assist users with low digital literacy.	Chatbot includes a welcome message with usage tips (e.g., "Ask me how to save!").
NFR-6	Scalability	The system should support up to 1,000 concurrent users without performance degradation.	The website should handle 1,000 users accessing the dashboard simultaneously.
NFR-7	Scalability	The database should handle up to 10,000 user records efficiently.	MySQL database can store and retrieve data for 10,000 users without delays.
NFR-8	Maintainability	The system should allow for easy updates and bug fixes without affecting core functionality.	Follow modular programming in Flask for easy code modifications.

NFR-9	Compatibility	The web application should be accessible on all modern browsers (Brave, Chrome, Firefox, Edge).	The website should work consistently across these browsers.
NFR-10	Compatibility	The web application should be accessible on Windows, macOS, and mobile devices.	The website should render properly on Windows, macOS, and mobile browsers.
NFR-11	Data Integrity	The system should prevent duplicate entries and ensure accurate data storage.	MySQL database should have unique constraints on critical fields (e.g., email).

2.1.2 Feasibility Study

The feasibility study of a project evaluates the practicality of developing and implementing WealthWise, focusing on technical and economic aspects to ensure its viability. Below is a general overview tailored to a broad context, adaptable to various project needs.

2.1.2.1 Technical Feasibility

Technical feasibility determines if existing technology supports development and implementation. The project utilizes free tools like Flask for the backend, MySQL for the database, Bootstrap for the front-end interface, and Chart.js, Matplotlib, Seaborn for data visualization, along with Bcrypt and OllamaLLM all of which are open-source and widely supported technologies suitable for a web-based financial management system. Overall, the available technology adequately supports WealthWise's development and initial deployment.

2.1.2.2 Economic Feasibility

Economic feasibility analyzes cost effectiveness and financial restrictions. WealthWise uses free technologies and local hosting to accommodate a student budget and Nepal's economic situation. However, financial constraints include the lack of funds for premium services such as cloud hosting or paid APIs for advanced features (e.g., bank integration), restricting deployment to local servers. The system's cost-effectiveness is increased by its free user access, which is in line with Nepal's economic situation where many people cannot purchase paid financial instruments. The current scope is financially feasible within a student budget, even though future scalability could necessitate investment.

2.1.2.3 Legal Feasibility

The legal feasibility study looks at restrictions imposed by the law that need to be closely adhered to. The use of Bcrypt for password hashing for authentication satisfies fundamental security criteria for web-based systems handling user data, which aids in compliance with Nepal's Electronic Transactions Act, 2063 (2008), which requires privacy and data protection. Full compliance with international standards like GDPR or ISO 27001 (aspirational goals) is challenging due to limited resources. There are no intellectual property issues, as the 50/30/20 budgeting method is a public concept, and all technologies used are open-source or licensed appropriately.

2.1.2.4 Operational Feasibility

The operational feasibility assesses whether end-users can efficiently use and maintain a system. The user-friendly interface ensures usability. The AI chatbot provides guidance, though users need basic internet access, which may limit rural adoption. Maintenance is feasible as a solo project but long-term support requires future planning. Overall, the system is operationally viable for its intended users with these considerations.

2.1.2.5 Schedule Feasibility

Schedule feasibility assesses the time-related structure and restrictions. WealthWise's 6-month timeline, which includes Spiral Model cycles for planning, risk analysis, development, testing, and evaluation, is manageable for a single developer. The schedule is feasible with focused effort, though advanced features (e.g., mobile app) are deferred to future iterations due to time limits.

2.1.3 Structured Modelling

Structured modelling is a critical phase in the system analysis process, where the functional requirements of a software are translated into visual representations to facilitate understanding, design, and implementation. This section includes Data Flow Diagrams (DFDs) as the primary modelling technique, supplemented by a Context Diagram to define the system boundary and prepare the groundwork for subsequent diagrams such as DFD Level 2, Entity-Relationship Diagram (ERD), and System Flow or Activity Diagram. The goal is Understanding data flow, system operations, and external interactions.

2.1.3.1 System Overview

A web-based financial management tool called WealthWise was created to combat excessive spending and increase financial literacy among a wide range of Nepali users. The system's goal is to give a user-friendly platform that allows for automated budgeting using the 50/30/20 rule, manual income and cost tracking, interactive chart visualization of spending trends, AI-powered financial guidance, email notifications, and offline report generation. Due to time and financial limitations, it does not include sophisticated features like bank integration or a mobile app, but rather focuses on essential features created with Flask for the backend, MySQL for data

storage, Bootstrap for the UI, and Chart.js, Matplotlib, and Seaborn for visualizations. Structured modeling, using Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERD), Context Diagrams, Activity Diagrams is employed to design, visualize, and document the system's data flow, ensuring clarity in how data moves between processes, data stores, and external entities, which is essential for a solo developer to manage complexity and communicate the system's structure.

1) DFD level 0:

The high-level overview of WealthWise's interactions with external entities is given by the Context Diagram (DFD level 0). It displays the data flow between WealthWise and its external entities and depicts the system as a single operation.

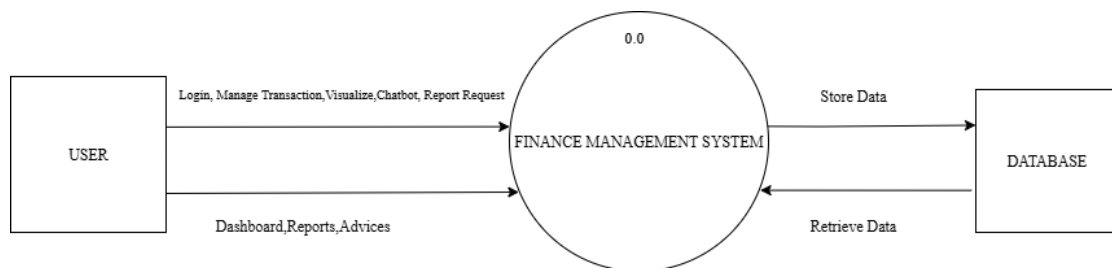


Figure 2.2: Context Diagram

Two external entities are shown in WealthWise's Data Flow Diagram (DFD): the database, which stores and retrieves data, and the user, who interacts with the system to manage funds. Data flows include the user sending the WealthWise System login information, transaction data, chatbot responses, and report requests. The WealthWise System then sends the user dashboards and reports. In order to support its operations, the WealthWise System also retrieves saved transaction and budget data and maintains user information, transactions, and budget modifications in the database.

2) DFD Level 1:

WealthWise is divided into key processes by DFD Level 1, which also illustrates the data flows between these activities and external organizations or data repositories.

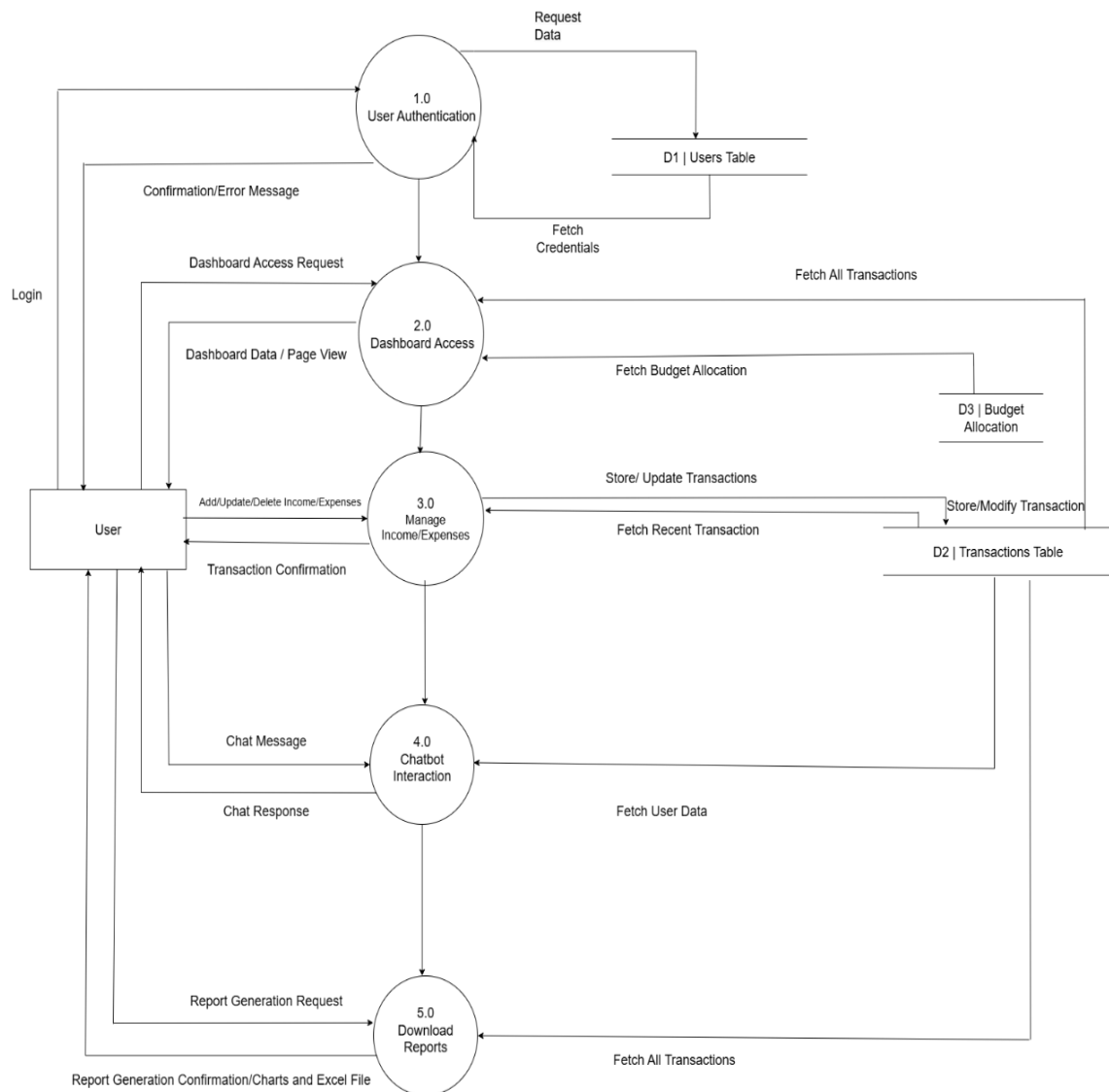


Figure 2.3: DFD Level 1

WealthWise's Level 1 Data Flow Diagram (DFD) lists five essential procedures: 2.0 Dashboard Access (showing financial data), 3.0 Manage Income/Expense (recording and updating income/expense data), 4.0 Chatbot Interaction (offering AI-driven advice), 5.0 Download Reports (creating offline Excel and chart reports), and 1.0 User Authentication (managing registration, login, OTP, and access control). The external entity, User, interacts with these processes by sending login credentials and transaction data to 1.0, queries to 4.0, and report requests to 5.0, receiving dashboards from 2.0, confirmations from 3.0, responses from 4.0, and reports from 5.0. Data stores include D1: Users Table (storing credentials), D2: Transactions Table (recording transactions), and D3: Budget Allocation (holding 50/30/20 budget data), with data

flows involving the User sending login details to 1.0 (fetching from D1), 3.0 updating D2 and D3 with income/expense data (reflected in 2.0 via D3), 4.0 querying D2/D3 for chatbot responses, and 5.0 generating reports from D2 and D3 for the User.

3) DFD Level 2:

A Level 2 Data Flow Diagram (DFD) offers a thorough breakdown of a particular process from the Level 1 DFD into more specific subprocesses.

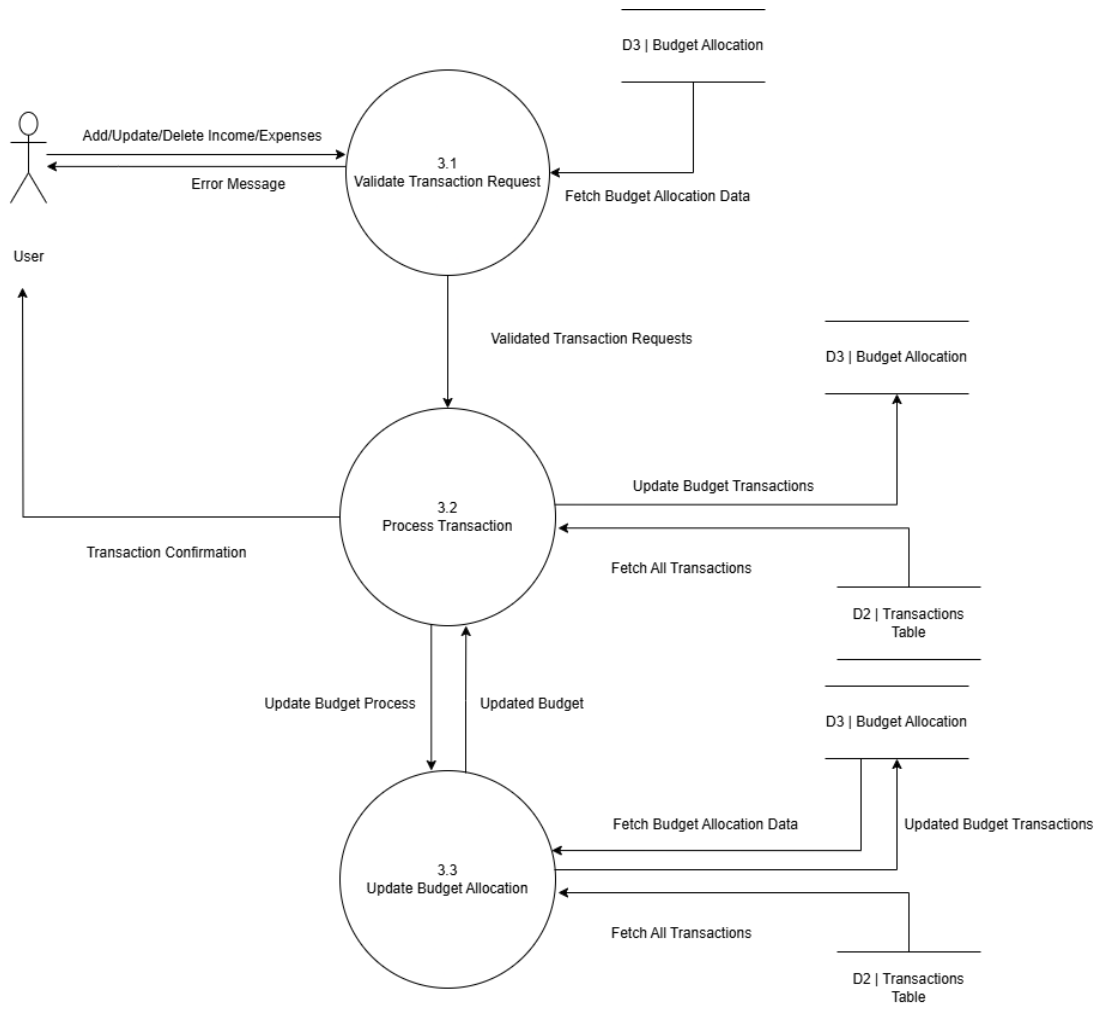


Figure 2.4: DFD Level 2

The DFD Level 2 for the 3.0 Manage Income/Expenses process in WealthWise details three subprocesses: 3.1 Validate Transaction Request (validates user inputs, fetching budget data from D3 Budget Allocation and sending error messages or validated requests), 3.2 Process Transaction (handles adding/updating/deleting transactions, updating D2 Transactions Table, sending confirmations to the User, and passing updates to D3 Budget Allocation), and 3.3 Update Budget Allocation (calculates and updates budget data in D3 using transaction data from D2, though incorrectly loops back to D2). The external entity, User, initiates transactions, while data stores include D2 Transactions Table (storing transaction records like id, user_id,

amount, date) and D3 Budget Allocation (holding budget percentages and totals), with data flows facilitating validation, processing, and budget updates across these components.

2.1.3.2 Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) illustrates the database structure, showcasing the entities, their attributes, and the relationships between them. To ensure effective data management for the system's features, including user authentication, transaction monitoring, budget allocation, and report production, this diagram is essential for comprehending how data is arranged and stored in the MySQL database. By offering a formal framework for storing financial data, the ERD supports WealthWise's mission to increase financial literacy among a varied population in Nepal. The ERD looks like this:

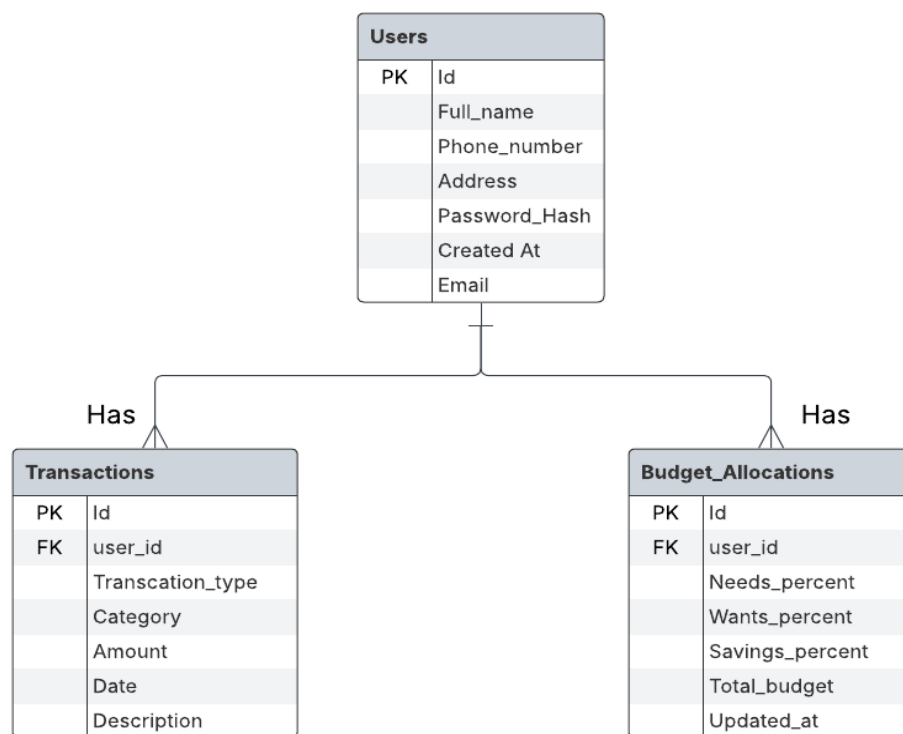


Figure 2.5: Entity-Relationship Diagram

The Entity-Relationship (ER) diagram for WealthWise defines three main entities which are Users, Transactions, and Budget_Allocations. Relationships include a One-to-Many relationship from Users to Transactions (via UserID, allowing one user to have multiple transactions like income of RS 2,000 or expense of RS 1000), and a One-to-Many relationship from Users to Budget_Allocations (via UserID, enabling multiple monthly budgets like Needs: RS 500, Wants: RS 300, Savings: RS 200), with Transactions indirectly influencing Budget_Allocations through User data to support the 50/30/20 budgeting rule.

2.1.3.3 System Flow Diagram

The System Flow Diagram provides a high-level representation of the operational flow of the system, illustrating the sequence of steps and decision points that users encounter while interacting with the system.

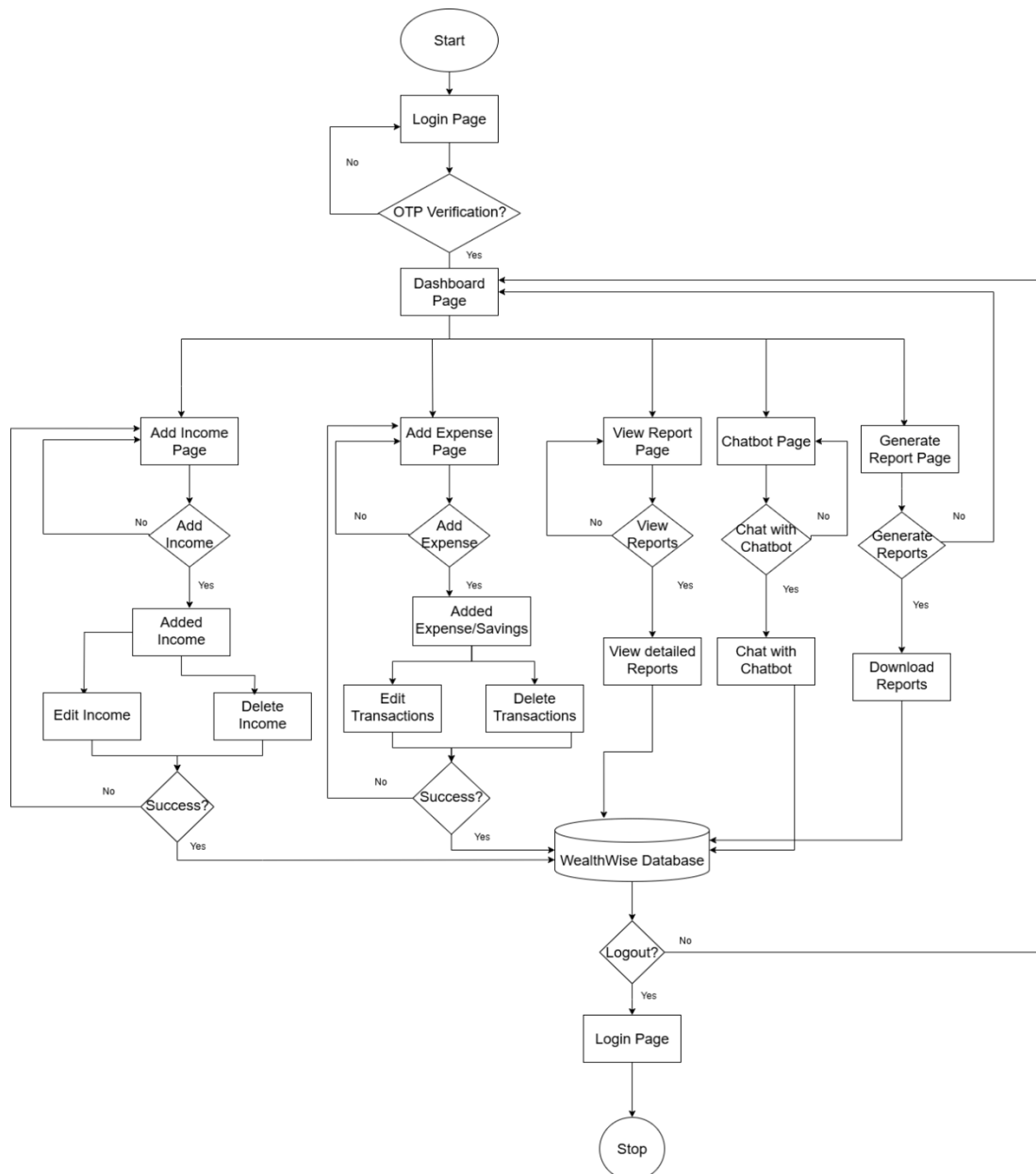


Figure 2.6: System Flow Diagram

The System Flow Diagram for WealthWise illustrates a straightforward user workflow starting with a login page, leading to a dashboard upon successful authentication. From the dashboard, users can navigate to add, edit, or delete income/expense transactions, view detailed reports, interact with a chatbot for advice, or download reports, with decision points determining each action. All processes

interact with the WealthWise database to store and retrieve data, providing a clear, sequential path for managing finances.

2.1.4 System Requirements Specification (SRS)

Table 2.12: System Requirements Table

Hardware Requirements		
Category	Requirement	Specification
Client Processor	Minimum: Intel Core i3	Ensures smooth operation for users on basic devices.
	Recommended: Intel Core i5/i7	Improves performance for users with modern laptops.
Client RAM	Minimum: 4GB	Ensures smooth navigation and basic processing.
	Recommended: 8GB or higher	Supports advanced features like real-time charts.
Client Storage	Minimum: 128GB SSD	Required for local caching and offline use.
	Recommended: 256GB SSD	Allows storage of offline reports and data.
Network	Minimum: 10 Mbps	Ensures stable connection for web-based access.
	Recommended: 100 Mbps or higher	Supports faster data transfer for multiple users.
Display	Minimum: 1366x768 resolution	Provides basic user interface experience.
	Recommended: Full HD (1920x1080)	Enhances user interface for better visualization.
Software Requirements		
Category	Requirement	Specification
Operating System	Windows 10/11	Compatibility with development tools (e.g., Flask, VS Code).
Database	MySQL 8.0	Depends on data structure and scalability needs for users, transactions, and budgets.
Back-end Technology	Flask 3.1.0	Handles server-side logic and API integration for WealthWise.
Front-end Technology	HTML, CSS, Bootstrap 5.3	Provides a user-friendly interface with responsive design.
Web Server	Localhost via Flask	Hosts the application locally due to budget constraints.

Version Control	Git & GitHub	Facilitates code management and version tracking for solo development.
Security Tools	Bcrypt	Ensures data security and user privacy for authentication and transactions.

2.2 Design

The design phase of a system focuses on creating a robust and scalable framework to support its financial management functionalities. This phase establishes the system's architecture and detailed components, ensuring efficient interaction between users and the application.

2.2.1 System Architecture

The system architecture of WealthWise follows a Client-Server architecture with a 3-Tier Architecture design pattern, suitable for a web-based financial management system. In the Client-Server model, the client which is the user's browser sends requests to the server hosted locally, which processes them and returns responses (e.g., dashboard data, charts). The 3-Tier Architecture divides the system into:

- i) **Presentation Layer:** The client-side interface (HTML, Bootstrap 5.3) displays data and interacts with users, ensuring an intuitive experience for diverse audience.
- ii) **Application Layer:** The Flask 3.1.0 backend handles business logic (e.g., budget calculations, transaction management) and communicates with the database.
- iii) **Data Layer:** MySQL 8.0 stores and retrieves data (e.g., Users, Transactions, Budgets), supporting scalability.

2.2.2 Database Design

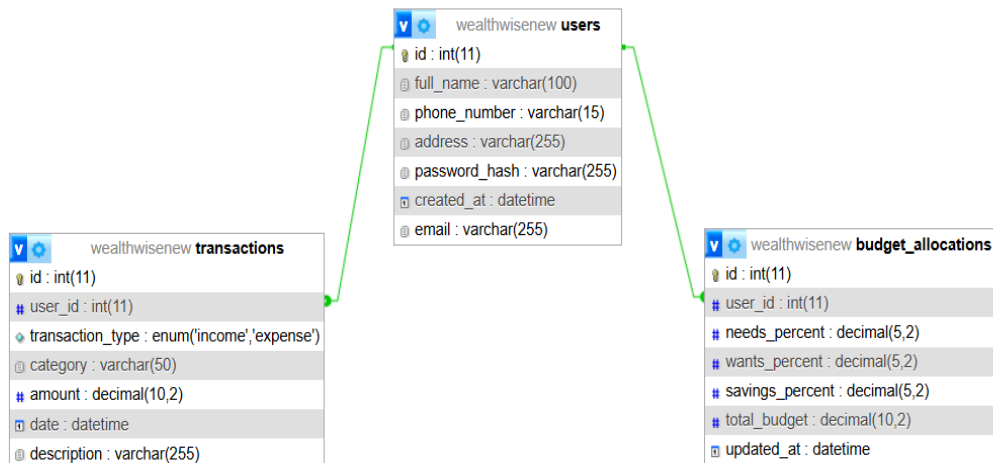


Figure 2.7: Database Design

2.2.2.1 Database Schema

The WealthWise database schema, designed in MySQL 8.0, comprises three tables: Users (with UserID as PK, FullName, PhoneNumber, Address, PasswordHash, CreatedAt, and unique Email), Transactions (with TransactionID as PK, UserID as FK, TransactionType, Category, Amount, Date, and Description), and Budget_Allocations (with BudgetID as PK, UserID as FK, NeedsPercent, WantsPercent, SavingsPercent, TotalBudget, and UpdatedAt). The schema enforces a One-to-Many relationship from Users to Transactions and Users to Budget_Allocations via UserID foreign keys, ensuring each user can have multiple transactions and budget entries, with constraints like unique Email and referential integrity maintained across tables.

2.2.2.2 Normalization

The database design employs normalization to avoid redundancy and ensure data integrity:

i. **First Normal Form (1NF):**

All tables have primary keys (e.g., UserID, TransactionID) and no repeating groups, with atomic attributes (e.g., Amount is a single value).

ii. **Second Normal Form (2NF):**

Tables are in 1NF, and non-key attributes are fully dependent on the primary key. For example, in Transactions, Category and Amount depend on TransactionID, not partially on UserID which is the foreign key.

iii. **Third Normal Form (3NF):**

Non-key attributes are independent of other non-key attributes. In Users, PasswordHash depends only on UserID (PK), not on FullName or Email. In Transactions, no non-key attribute depends on another (e.g., Category doesn't determine Amount). In Budget_Allocations, NeedsPercent depends on BudgetID, not on WantsPercent. This satisfies 3NF. Redundancy is minimized, as user details are stored once in Users and linked via UserID, reducing update anomalies and supporting efficient storage for up to 10,000 records.

2.2.3 UI Design:

The UI design for WealthWise prioritizes usability and accessibility for a diverse audience in Nepal, adhering to UX principles like simplicity, consistency, and feedback. The UI is designed to be user friendly and responsive, using Bootstrap 5 for consistency. The base template (base.html) features a sidebar with navigation links (Dashboard, Transactions, Chatbot, Generate Reports, Logout) and a main content area styled with custom CSS (static/css/style.css). The dashboard (db.html) displays financial metrics in cards (Total Income, Total Expenses, Needs/Wants/Savings) and

Chart.js visualizations (pie and bar charts). The design ensures accessibility across devices, with a focus on clear visuals (e.g., color-coded cards: green for income, red for expenses).

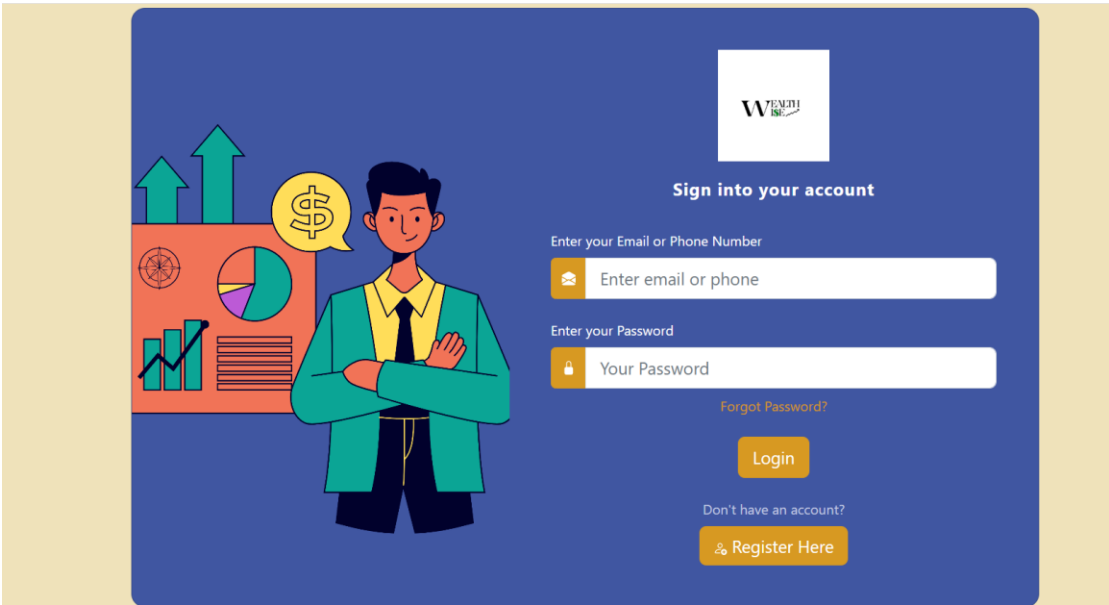


Figure 2.8: Login Page



Figure 2.9: Dashboard Page

Welcome, Kaushal Bikram Gajmer

Add Income

Amount (Rs)

Category

Date

Description

[Add Income](#)

[Back to Dashboard](#)

Recent Incomes
 No recent incomes found

Figure 2.10: Add Income Page

Welcome, Kaushal Bikram Gajmer

Add Expense

Understanding Needs vs. Wants
 Needs are essential expenses for normal (e.g., rent, groceries, utilities).
 Wants are non-essential expenses for enjoyment (e.g., dining out, entertainment). We follow the 50/30/20 rule: 50% of your income for needs, 30% for wants, and 20% for savings.
Examples:
 Needs: Rent, groceries, transportation, utilities.
 Wants: Dining out, movies, subscriptions, shopping.

Current Month Budget Status
 Needs Budget: Rs 0.0 / Rs 100.0 Wants Budget: Rs 0.0 / Rs 100.0
 Remaining: Rs 100.0 Remaining: Rs 100.0
 Budget status updates in real-time as you add expenses.

Amount (Rs)

Description

Category

Date

[Add Expense](#)

[Back to Dashboard](#)

Recent Transactions (This Month)
 No recent transactions found for this month.

Figure 2.11: Add Expense Page

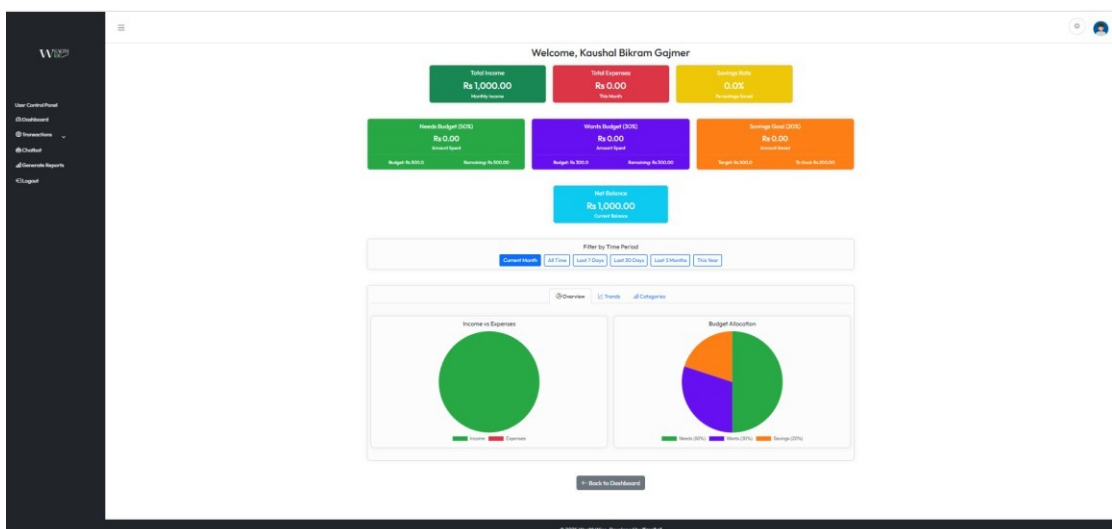


Figure 2.12: View Report Page

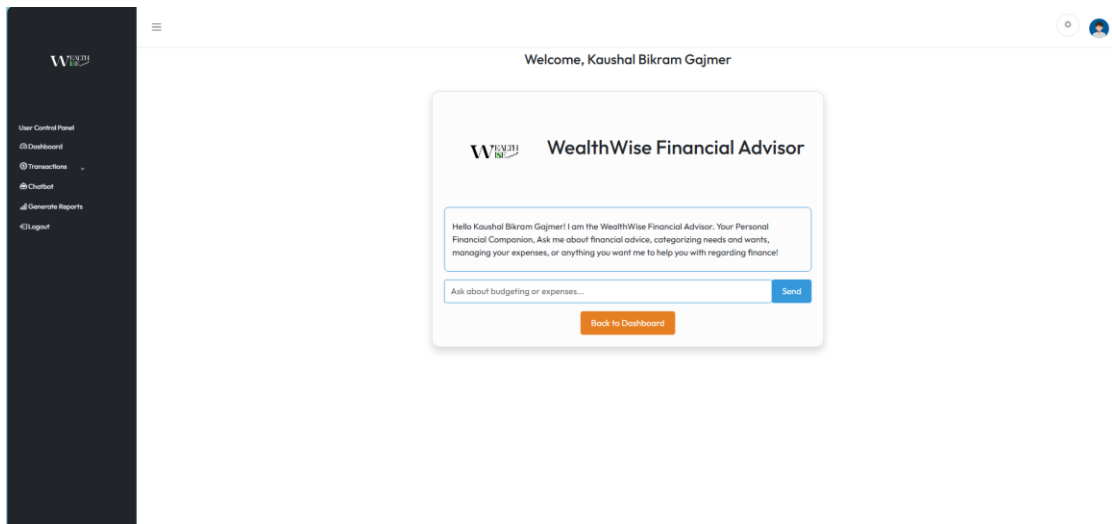


Figure 2.13: Chatbot Page

2.2.4 Algorithm Design

WealthWise implements essential algorithms to support its core functionalities, focusing on budget allocation, transaction management, and user engagement. Among these, the most critical include Budget Calculation, Transaction Validation, Budget Threshold Notification, Chatbot Financial Advice, and Report Generation Algorithms.

1. Budget Calculation Algorithm

Allocates funds using the 50/30/20 rule when the dashboard loads, if income exists.

```

IF monthly_income > 0 THEN
  SET needs_limit = 0.50 * monthly_income
  SET wants_limit = 0.30 * monthly_income
  SET savings_limit = 0.20 * monthly_income
  INSERT OR UPDATE budget_allocations (user_id, needs_limit, wants_limit,
  savings_limit)
END IF
  
```

2. Transaction Validation Algorithm

Ensures secure and accurate transaction processing.

```

IF NOT user_id IN session THEN
  RETURN "Unauthorized"
END IF
IF NOT (amount AND category AND date) THEN
  RETURN "Missing fields"
END IF
IF transaction_id THEN
  UPDATE transactions SET amount, category, date WHERE id = transaction_id
ELSE
  INSERT INTO transactions (user_id, type, category, amount, date)
  
```

END IF

3. Budget Threshold Notification Algorithm

Monitors spending and sends email alerts if needs or wants exceed 80% of budget.

```
IF category IN ("needs", "wants") AND monthly_income > 0 THEN
  SET limit = (monthly_income * budget_percent) / 100
  SET spent = SUM(transactions.amount) WHERE category = category
  IF (spent / limit * 100) > 80 THEN
    SEND_EMAIL(user.email, "Budget Alert", spent, limit)
  END IF
END IF
```

4. Chatbot Financial Advice Algorithm

Generates personalized advice using OllamaLLM based on transaction data.

```
SET financial_data = AGGREGATE(transactions, user_id)
SET prompt = FORMAT_TEMPLATE(template, financial_data, context,
user_message)
SET response = LLM_INVOKE(prompt)
RETURN {response, new_context}
```

5. Report Generation Algorithm

Summarizes monthly transactions for financial reports.

```
IF NOT user_id IN session THEN
  RETURN "Unauthorized"
END IF
SET total_income = SUM(transactions.amount) WHERE type = "income" AND
category != "savings"
SET expenses = SUM(transactions.amount) WHERE type = "expense"
SET net_balance = total_income - expenses
SET budget_limits = CALCULATE_BUDGET(total_income)
RENDER_REPORT(total_income, expenses, net_balance, budget_limits, transactions)
```

These algorithms collectively ensure that WealthWise delivers personalized, accurate, and actionable financial management for users in Nepal.

2.2.5 System Flowcharts

The system flowcharts show the key processes implemented in a system.

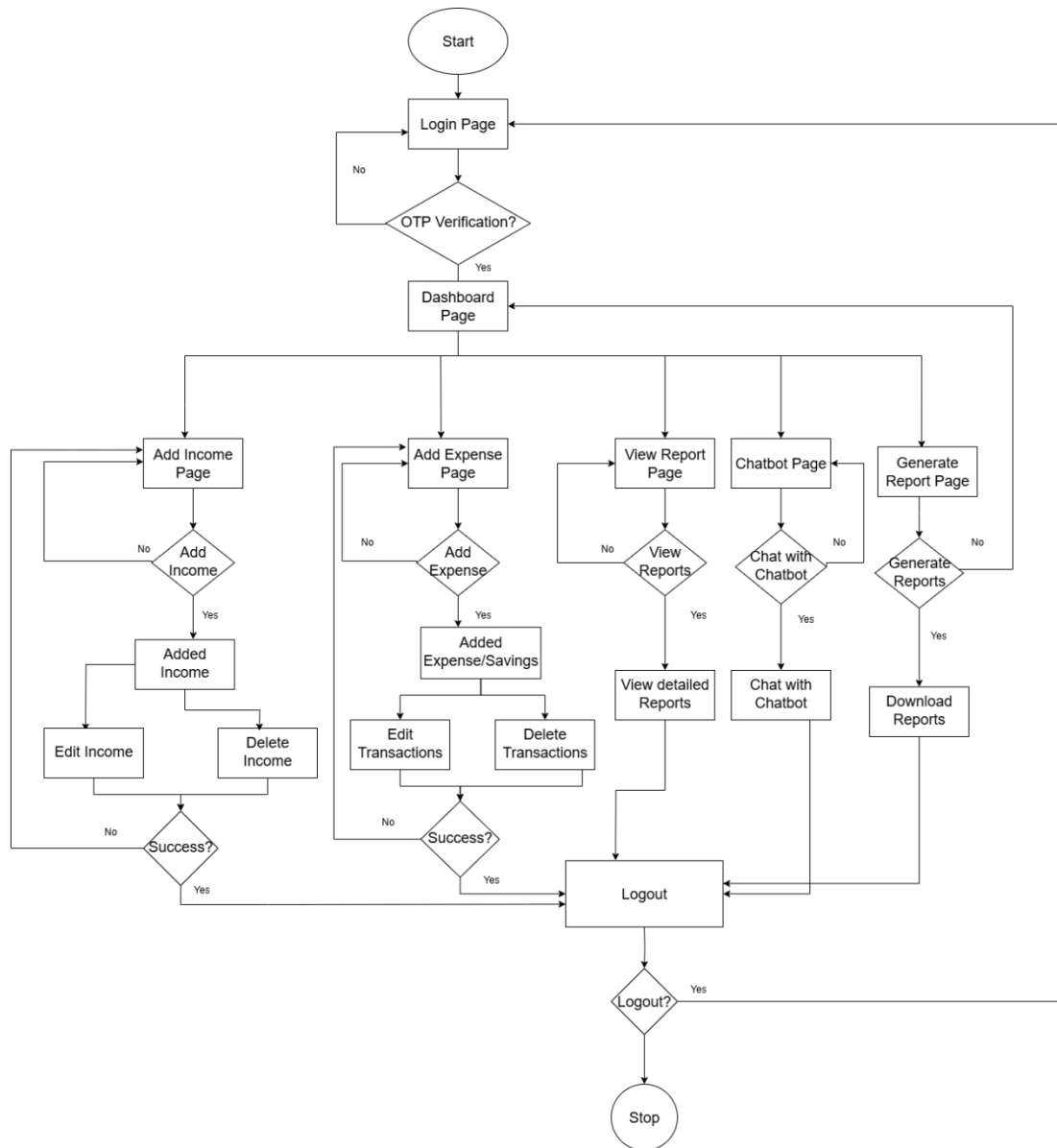


Figure 2.14: System Flowchart

The process begins with the Login Page, where users authenticate using email or phone number and password storing session data and sending a welcome email upon success, redirecting to the Dashboard. If there is a failed attempts then it loops back with errors. From the Dashboard, users can navigate to Add Income/Expense Pages to enter transactions, validated and stored in the Transactions table, with options to edit or delete, supporting manual entry. The system monitors budgets, calculating spending percentages and sending email warnings if needs or wants exceed 80%, as handled in the Dashboard route. Users can generate offline reports via the Visualize Page, fetching transaction data to create Excel files and bar/scatter charts, saved for offline access. Additionally, the Chatbot Page allows users to submit queries, fetching financial data

to provide AI-driven advice, updating the context for further interaction. Finally, when user can logout clearing their session data and redirecting to the login page.

2.3 Implementation

The implementation phase of a software project involves the practical application of design and planning into a functional system. Below is a general overview of the tools and development environment commonly utilized during this stage, providing flexibility for various project contexts.

2.3.1 Tools and Development Environment

Table 2.13: Tools/Technology used and its purpose

Tool/Framework	Category	Purpose
Bootstrap	CSS Framework	Designing responsive, mobile-friendly UI with pre-built components
Chart.js	JavaScript Library	Rendering interactive financial charts on the dashboard
CSS	Style Sheet Language	Styling HTML for consistent, userfriendly interface
Draw.io	Diagramming Tool	Creating DFDs and ERDs for system and MySQL database design
GitHub	Version Control	Hosting codebase, managing branches, pull requests, and issue tracking
HTML	Markup Language	Structuring web pages for login, dashboard, and transaction interfaces
JavaScript	Scripting Language	Enabling dynamic chatbot interactions and Chart.js visualizations
Mailpit	Email Testing Tool	Testing email notifications (e.g., welcome, OTP) during development
MySQL	RDBMS	Storing user data, transactions, and budget settings
Ollama with Llama3	AI Model Framework	Powering chatbot for financial advice based on user data
Pandas	Data Analysis Library	Structuring transaction data for reports and Excel exports
Python with Flask	Backend Framework	Handling server-side logic, routing, form processing, and database tasks
Matplotlib	Visualization Library	Generating static charts (e.g., expense breakdowns) for reports

Render	Cloud Hosting Platform	Deploying the application with managed PostgreSQL database
Seaborn	Visualization Library	Enhancing Matplotlib charts with statistical graphics for reports
Visual Studio Code	IDE / Code Editor	Writing and managing Python, HTML, CSS, and JavaScript code

2.3.2 Software/Hardware Setup Required for Development:

Table 2.14: Development Environment for WealthWise

Component	Specification	Remarks
Browser	Brave / Edge	Used for testing and debugging the web application
Database	MySQL (local), PostgreSQL (Render)	Stores user data, transactions, budgets; flask_mysqldb for MySQL
Frontend	HTML, CSS, Bootstrap, Chart.js, JavaScript	Enables responsive, interactive UI with financial visualizations
Backend	Python (v3.1+), Flask	Manages server-side logic, routing, form processing, and database tasks
Operating System	Windows 11	Compatible with Python, Flask, MySQL, and front-end development tools
Processor (CPU)	Intel Core i5-1350H, RTX 3050	Ensures efficient execution of development, build, and testing processes
RAM	16 GB	Sufficient for running MySQL, Flask server, and browser-based tools
IDE/Editor	Visual Studio Code	Supports Python, HTML, CSS, JavaScript with extensions for Flask, Git
Version Control	Git, GitHub	Facilitates code versioning, collaboration, and deployment tracking
Email Testing	Mailpit	Tests email notifications (e.g., welcome, OTP, alerts) during development
Package Manager	pip	Manages Python dependencies (e.g., Flask, Bcrypt, Pandas)

Other Tools	Ollama with Llama3, Draw.io	Powers AI chatbot for financial advice and creates system/database diagrams
--------------------	--------------------------------	--

2.3.3 System Modules and Their Implementation

In software development, a module is like a building block which contains a small, self-contained piece of a program that handles a specific task. Modules make it easier to design, build, test, and update software because they can be developed independently and connected through clear interfaces.

In WealthWise, modules are distinct components that manage different aspects of the financial management system, such as budgeting, expense tracking, or user interaction. Each module is designed to work on its own but integrates with others to create a seamless experience for users, whether they are students managing personal finances tracking revenue in Nepal. The modules used are:

a) Authentication Module (/login, /registration, /logout, /verify_otp, /forgot_password, /reset_password):

Handles user identity and access control with enhanced security.

Functionalities:

- i. User registration (full name, email, phone, address, Bcrypt-hashed password).
- ii. Login (email/phone/password with OTP verification via Mailpit).
- iii. Secure session management, logout, and password reset with token-based links.
- iv. OTP resending (5-minute expiry, lockout after 5 failed attempts).

b) 2FA Module (/toggle_2fa):

Enhances user experience by allowing users to toggle the OTP login feature on or off, addressing the inconvenience of mandatory OTP verification.

Functionalities:

- i. Enables/disables 2FA via a toggle in the user profile dropdown.
- ii. Manages 2FA state using session-based storage.
- iii. Integrates with login flow to require OTP verification only when 2FA is enabled, offering flexibility to skip OTP when disabled.

c) Dashboard Module (/dashboard/<user_id>):

Serves as the central hub for monthly financial overviews.

Functionalities:

- i. Displays monthly income, expenses, and 50/30/20 budget allocations (Needs, Wants, Savings).
- ii. Integrates Chart.js for pie and bar charts (expense breakdown, income vs. expense).
- iii. Triggers email warnings at 80%/90% budget thresholds.
- iv. Automates reminders to log income/expenses.

d) Transaction Management Module (/add_income, /add_expense, /edit_income, /delete_income, etc.):

Enables CRUD operations on monthly income and expense records.

Functionalities:

- i. Adds/updates/deletes income/expense transactions (amount, category, date, description) with Decimal precision.
- ii. Focuses on current-month data.
- iii. Checks budget limits (80%/90%/100%) post-transaction and sends email alerts.

e) Financial Advisor Chatbot Module (/chatbot/<user_id>):

Provides AI-driven financial advice based on monthly data.

Functionalities:

- i. Accepts user queries and retrieves monthly income, expenses, and budget status.
- ii. Uses Llama3 (via Ollama) to offer concise advice in NPR for Nepalese students.
- iii. Focuses on budgeting and savings.

f) Reporting and Visualization Module (/visualize/<user_id>):

Generates offline financial reports from all transactions.

Functionalities:

- i. Fetches all transaction data and creates Excel (.xlsx) summaries with a Summary sheet.
- ii. Generates PNG charts (expense distribution, monthly trends) using Matplotlib/Seaborn.
- iii. Saves files in an OfflineReports directory.

g) View Reports Module (/view_reports/<user_id>):

Allows review of current-month and historical transaction data.

Functionalities:

- i. Displays monthly totals (income, expenses, net balance) with 50/30/20 budget comparisons.
- ii. Provides JavaScript-filterable view of all transactions.

2.3.4 Coding Standards and Conventions

The development of WealthWise follows strict coding standards and conventions to ensure code readability, maintainability, and consistency, which are essential for a project that may require future updates or collaboration. These standards align with industry best practices. Wealthwise follows Python's PEP 8 guidelines, and are tailored to the Flask-based web application's architecture where code is written in Jinja 2 Template. By following structured naming conventions, comprehensive documentation practices, and robust coding best practices, the project maintains a high-quality codebase that supports efficient debugging, scalability, and security, meeting the non-functional requirements.

Thorough documentation ensures that the codebase is understandable and maintainable. Inline comments are used to explain complex logic. Additionally, a README.md file documents setup instructions, including steps to install dependencies (pip install flask flask-mysqldb, etc), configure MySQL, and run the app ensuring that the project can be easily set up by others or revisited later.

WealthWise follows best practices to ensure a robust and secure codebase. The application is structured modularly, with separate Flask routes for distinct functionalities (e.g., /login, /add_income, /chatbot, etc), promoting separation of concerns and ease of updates. Error handling is implemented using try-except blocks, where exceptions during database operations trigger mysql.connection.rollback() and a user-friendly flash message, preventing crashes and ensuring data integrity. In accordance with security best practices, database queries also employ parameterized statements (cursor.execute('SELECT * FROM users WHERE id=%s', (user_id,))) to stop SQL injection. Together, these procedures guarantee that WealthWise is scalable, safe, and maintained for Nepal's heterogeneous user base.

2.3.5 Security Considerations

Any project that aims to secure user data and maintain system integrity must have security, but this is especially true for financial management applications that handle sensitive data like transactions and personal information. In accordance with non-functional requirements and addressing potential risks in Nepal's context, such as inconsistent internet reliability and low user awareness of security procedures, this section describes the steps taken to address authentication, encryption, access control, and data validation. All of these safeguards work together to protect user accounts, stop illegal access, and preserve data integrity across the program.

i) **Authentication:**

Only authorized users are able to access WealthWise through authentication. Two login options are supported by the application: email and mobile number/password. Users provide their password and email address or phone number via the /login route, and these are verified against the users table. Flask-Bcrypt's check_password_hash function is used to verify passwords, guaranteeing a secure comparison with hashes that have been stored. The system also employs 2FA, which entails utilizing an OTP for phone or email login. Persistent user identification across requests is made possible by creating a session with session['user_id'] = account['id'] after successful authentication.

ii) **Encryption:**

Private information is shielded from unwanted access by encryption. In order to prevent plaintext passwords from ever being saved in the users table, WealthWise employs Flask-Bcrypt to hash passwords before to storage. This computationally demanding hashing technique complies with recommended standards for password security by thwarting brute-force assaults.

iii) Access Control:

By limiting users to their own data, access control stops illegal activity. Routes like /dashboard and /add_income employ the is_logged_in() function to reroute unauthenticated users to the login page by determining whether user_id is present in the session. By verifying the user_id of the session with the requested user_id parameter and rerouting to logout if unlawful, user-specific access is further enforced. This preserves privacy by preventing users from accessing the dashboards or transactions of others.

iv) Data Validation:

Data validation ensures the integrity and reliability of user inputs and database operations. In the /registration route, the RegistrationForm validates inputs like email and phone number, checking for duplicates preventing conflicts and flashing messages like flash ('Email Already Registered! Please use a new email', 'danger'). Transaction routes like /add_income and /add_expense enforce required fields, rejecting invalid submissions. SQL injection is mitigated by using parameterized queries throughout ensuring safe database interactions. Additionally, the /chatbot route sanitizes user messages with user_message.strip(), preventing malformed inputs from disrupting the LLM processing.

2.3.6 Testing and Debugging

To guarantee a system's dependability, functionality, and usability, testing and debugging are essential.

2.3.6.1 Unit Testing

To make sure all the function works as intended, unit testing includes examining the smallest functional units of code, such as functions, methods, or modules. Unit testing promotes modular design and increases test coverage, reducing the likelihood of defects in the final product.

In WealthWise, unit testing was applied to individual components, such as the Python functions handling the registration, login, 50/30/20 budgeting calculations, expense categorization, and chatbot response logic. For example, a unit test verified that the budgeting function correctly split an input income (e.g., Rs 10,000) into needs (Rs 5,000), wants (Rs 3,000), and savings (Rs 2,000). The following table lists the test cases for unit testing:

Table 2.15: Unit Testing Test Cases

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Test Status
UT-001	User Registration with invalid credentials	1.Navigate to the Registrati on Page 2. Fill all the required fields 3. Click on Register Button	Username: 12345 Email: 1312313 Phone: 12321 Address: 1231322 Password: 1234 Confirm Password: 1234	User should not be able to register	User is not able to register and error message s are shown	Pass
UT-002	User Registration with existing phone number	1. Navigate to the Registrati on Page 2. Register with existing phone number 3. Click on Register Button	Username: Kaushal Email: kaushalbikram44@gmail.com Phone: +9779764233148 (existing) Address: Hattiban Password: Password@123 Confirm Password: Password@123	User should not be able to register	User is not able to register and error message is shown	Pass
UT-003	User Login with valid credentials	1. Enter valid username 2. Enter valid password 3. Click Login	Email: 021bim013@sxc.edu.np Password: Password@123	User should be able to login to the system and navigate d to the	User is shown a success message navigate d to the dashboar d page	Pass

				dashboard page		
UT-004	Budget Calculation with valid income	1. Call budgeting function 2. Input income 3. Verify allocation	Amount: 1000 Category: Salary Date: Today's Date	Budget should split as: Needs (500), Wants (300), Savings (200) as per the 50/30/20 rule.	Budget splits as expected	Pass
UT-005	Budget Calculation with invalid income	1. Log in 2. Navigate to Add Income Page 3. Enter invalid income (e.g., negative or non-numeric) 4. Submit	Amount: -1000 Category: Salary Date: Today's Date	Error message should be displayed	Error message is shown	Pass
UT-006	Expense Categorization with valid input	1. Navigate to Add Expense 2. Select a category from dropdown 3. Submit expense 4. Check result	Amount: 2000 Description: Groceries Category: Needs	Category should be saved as Needs and dashboard should be updated	Category saved as Needs and the dashboard is updated	Pass

UT-007	Chatbot Response for budget query	1. Call chatbot function 2. Submit query 3. Verify response	Query: How to budget better?	Response < 100 words with budgeting tips	Response provides budgeting tips within limit	Pass
--------	-----------------------------------	---	------------------------------	--	---	------

Test case 1: User Registration with invalid credentials.

Expected Output: Test is unsuccessful if invalid data is entered.

Evidence:

The screenshot shows a registration form with the following fields and errors:

- Full Name:** Input is "12345". Error message: "Name cannot should have only letters and spaces".
- Email Address:** Input is "1312313". Error message: "Invalid email address".
- Phone Number:** Input is "12321". Error message: "Enter a valid phone number (10-15 digits, country code optional)".
- Address:** Input is "1231322".
- Password:** Input is "Your Password Here". Error message: "Password must be at least 8 characters with uppercase, lowercase, digits, and special characters".
- Confirm Password:** Input is "Confirm Password".

A "Register" button is located at the bottom right of the form.

Figure 2.15: Invalid Registration

Conclusion:

Users can register with valid credentials only.

Test case 2: Registration Verification with existing phone number.

Expected Output: Test is successful if existing phone number is used.

Evidence:

The figure consists of two screenshots of a web application's 'Create your account' page. The top screenshot shows a successful registration with the following details: Full Name: Kaushal Bikram Gajmer, Email Address: kaushalbikram44@gmail.com, Phone Number: +9779764233148, Address: Hattiban, Lalitpur. A password strength indicator shows requirements: At least 8 characters, At least one uppercase letter, At least one lowercase letter, At least one number, and At least one special character (e.g., @\$!%*?&). The bottom screenshot shows an error message: 'Phone Number Already Registered! Please use a new number'. The registration form fields are: Full Name (placeholder: Enter your name), Email Address (placeholder: Enter your email), and Phone Number (placeholder: +977 1234567890). A note below the email field states: 'Valid email format (e.g., user@example.com)'.

Figure 2.16: Invalid Registration with Existing Phone Number

Conclusion:

Users can register with phone numbers only.

Test Case 3: User Login Verification

Expected Output: Test is successful if valid data is entered.

Evidence:

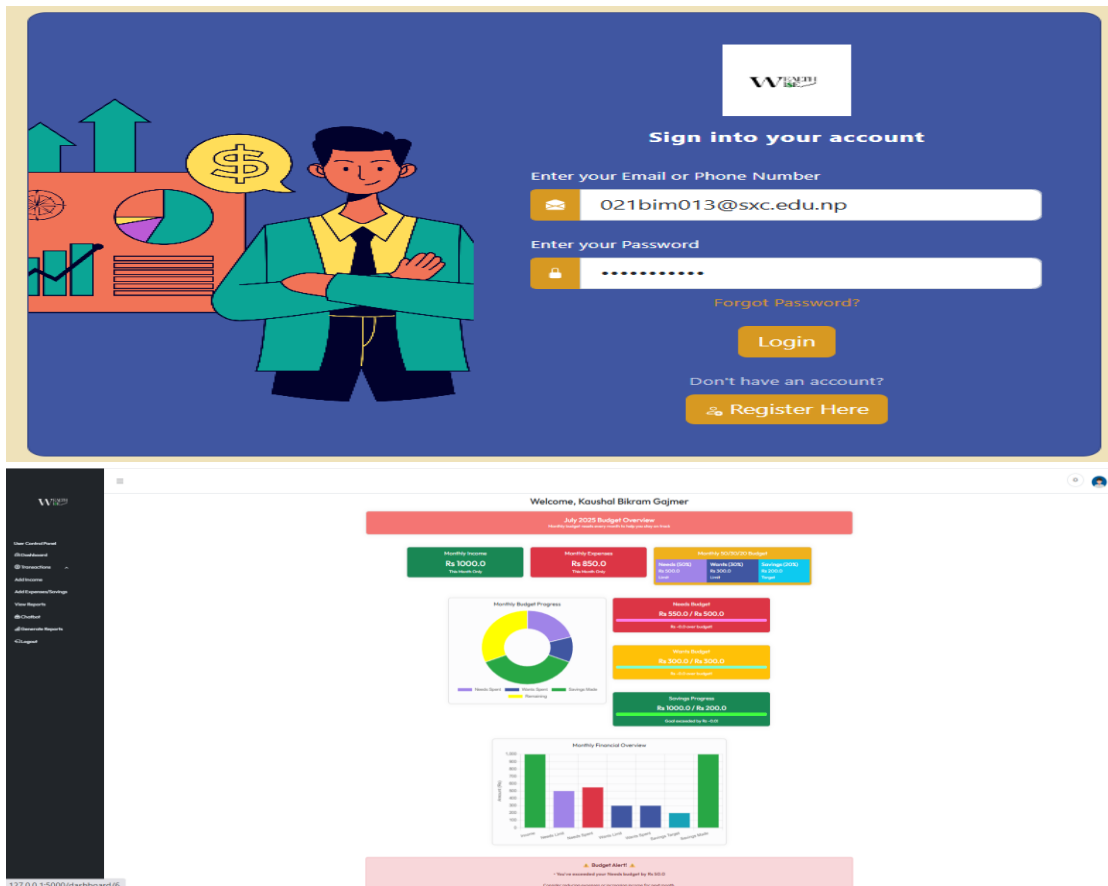


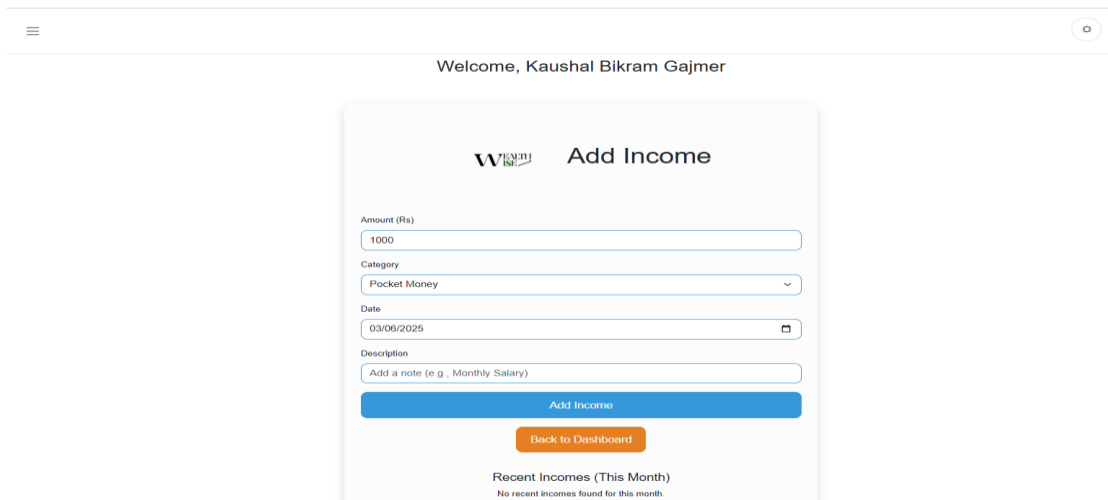
Figure 2.17: Valid Login

Conclusion: Only registered user and password are granted access.

Test Case 4: Budget Calculation with valid income.

Expected Output: Test is successful if valid income is provided.

Evidence:



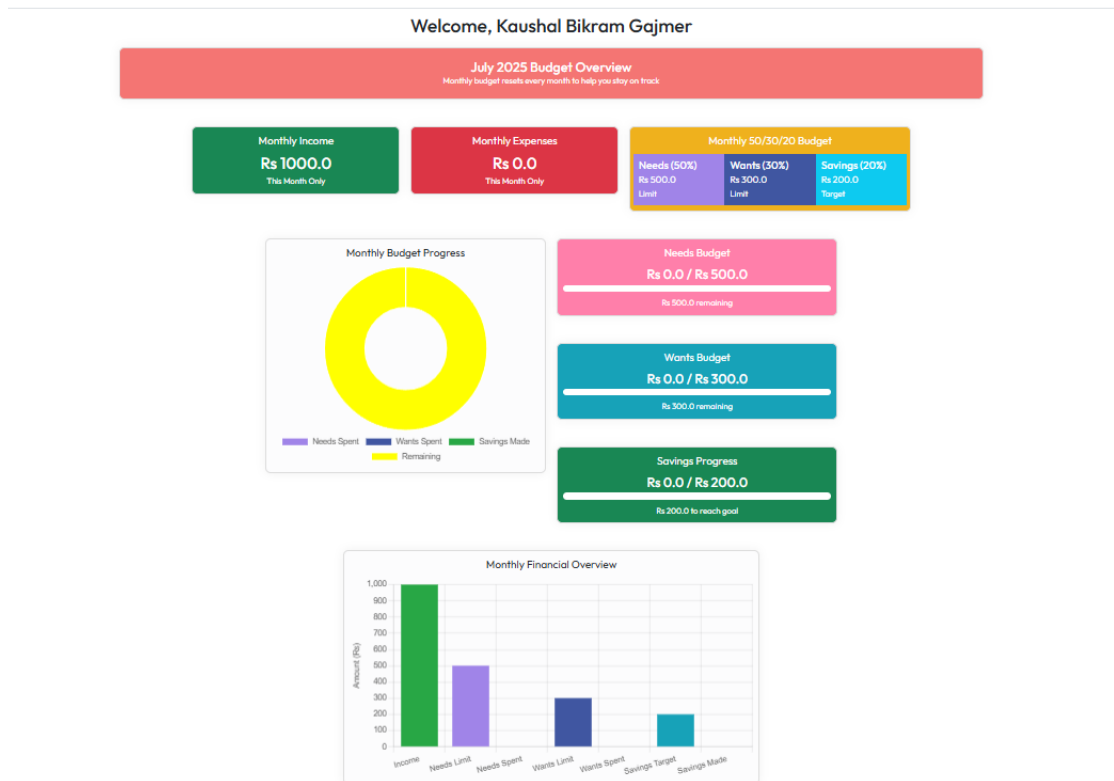


Figure 2.18: Budget Calculation with Valid Income

Conclusion: Budget is calculated correctly using the 50/30/20 rule.

Test Case 5: Automatic Budget Allocation Verification

Expected Output: Test is successful if invalid income is entered

Evidence:

Welcome, Kaushal Bikram Gajmer

W7EALTH **Add Income**

Amount (Rs)
-1000

Category
Select a category

Date
03/02/2025

Description
Add a note (e.g., Monthly Salary)

Add Income

Back to Dashboard

Amount must be a positive number

Figure 2.19: Budget Calculation with invalid income

Conclusion: Add income page only accepts valid data.

Test Case 7: Chatbot Response for budget query

Expected Output: Test is successful if a relevant response is generated.

Evidence:



Figure 2.20: Chatbot Response Verification

Conclusion: Chatbot provides concise and relevant budgeting advice.

2.3.6.2 Integration Testing

Integration testing is the type of testing that evaluates how individual components or modules work together when combined. It focuses on identifying issues at the interfaces between modules, such as data mismatches or communication failures, ensuring seamless interaction across the system.

For WealthWise, integration testing verified the interactions between modules, such as the Expense Tracking Module saving transactions to the Database, which the Visualization Module then used to generate charts. The Integration Test Test Cases include:

Table 2.16: Integration Testing Test Cases

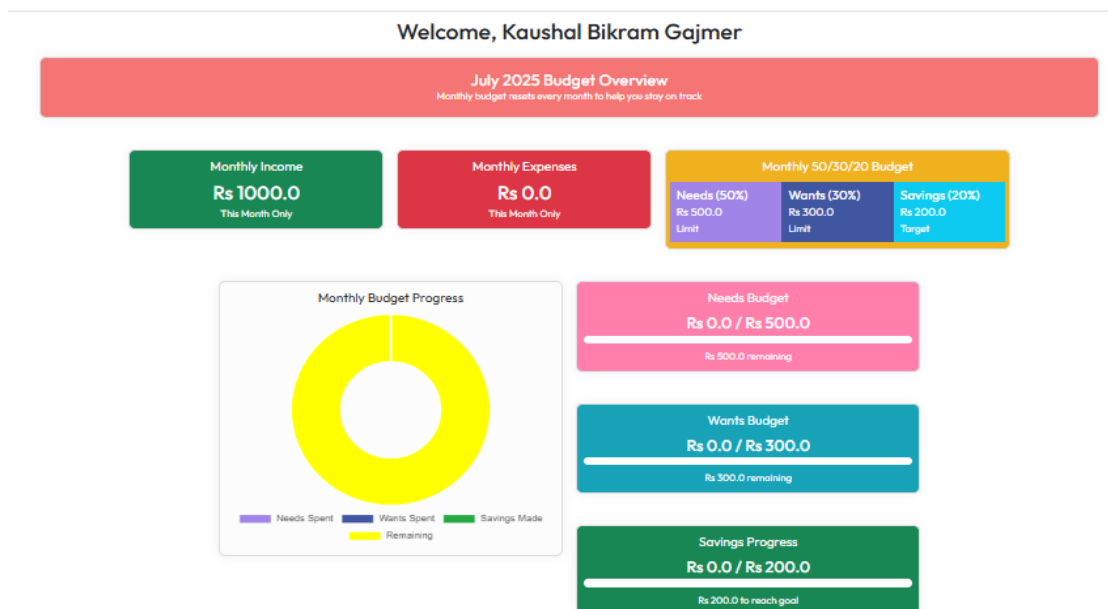
Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Test Status
IT-001	Expense Tracking to Visualization	1. Add expense 2. Check chart generation	Amount: 200 Category: wants	Chart should update to new expense	Chart is updated correctly	Pass

IT-002	2FA Toggle with Login Flow	1. Log in 2. Enable 2FA via toggle 3. Log out 4. Log in again 5. Disable 2FA via toggle 6. Log out 7. Log in again	Email: 021bim013@sxc.edu.np Password: Password@123	OTP should be required when 2FA is enabled. No OTP should be required when OTP is disabled.	OTP is asked with 2FA on No OTP is asked with 2FA off	Pass
--------	----------------------------	--	---	--	--	------

Test Case 1: Expense Tracking to Visualization

Expected Output: Test is successful if expense data is reflected in charts.

Evidence:



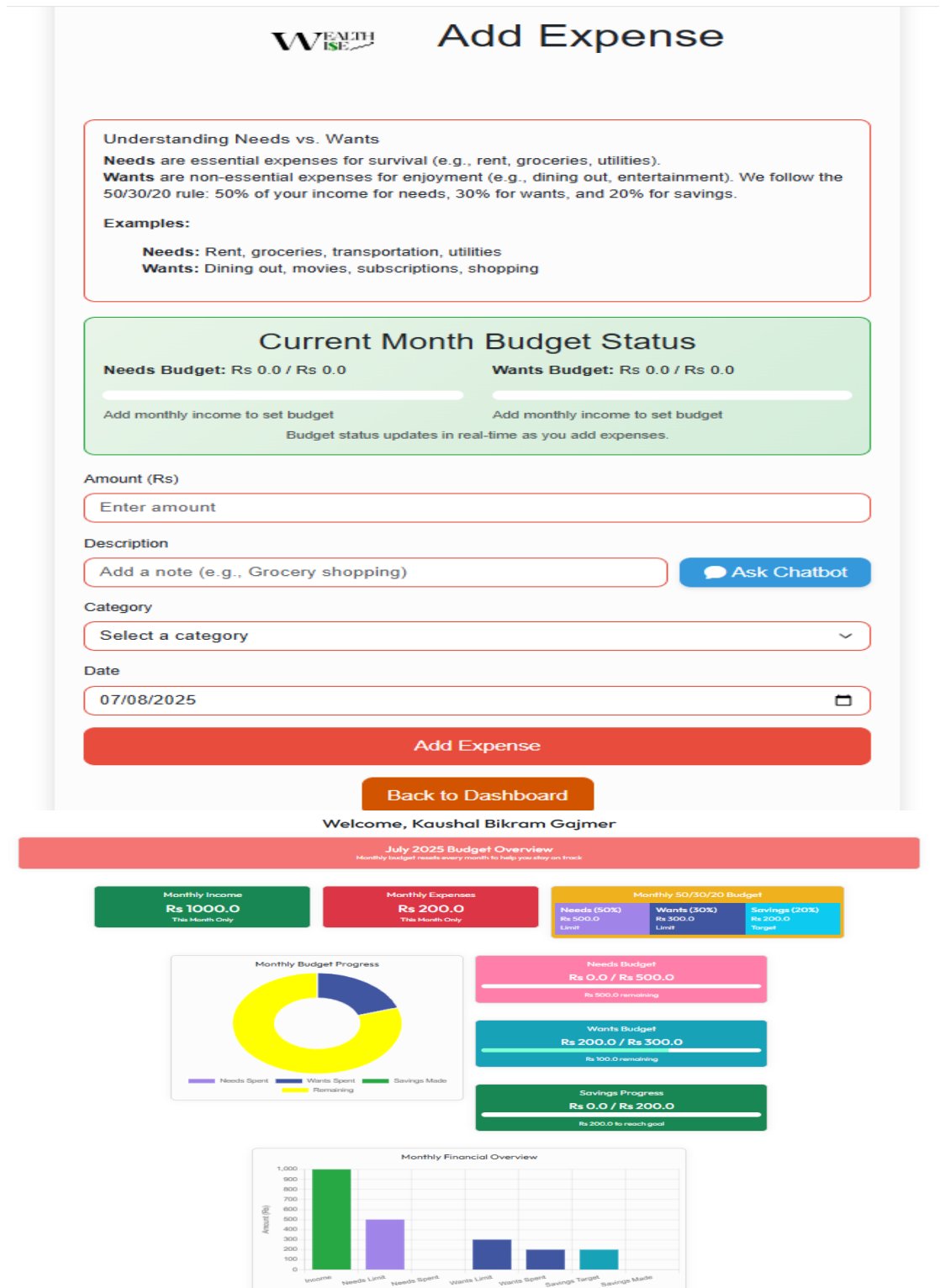


Figure 2.21: Chart update with Database

Conclusion: Charts are updated correctly with new expense data.

Test Case 2: 2FA Toggle with Login Flow

Expected Output: Test is successful if OTP is requested only when 2FA is enabled and skipped when disabled.

Evidence:

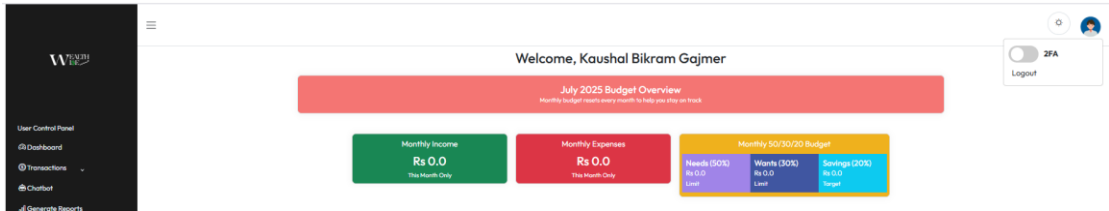
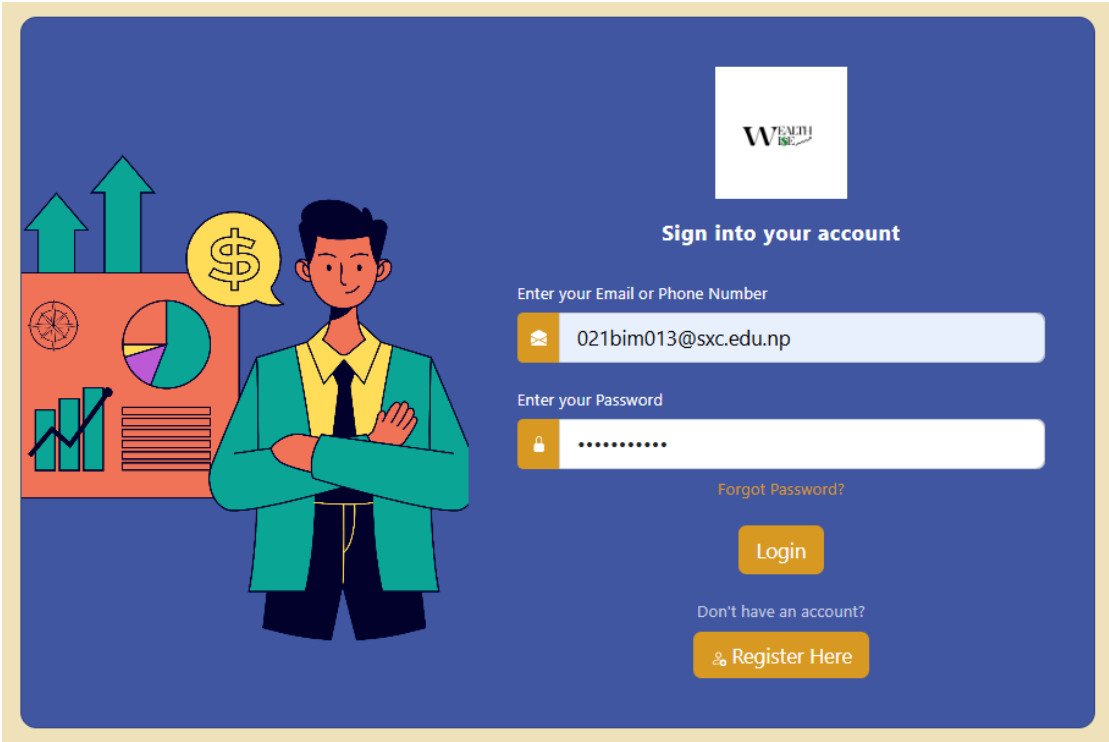
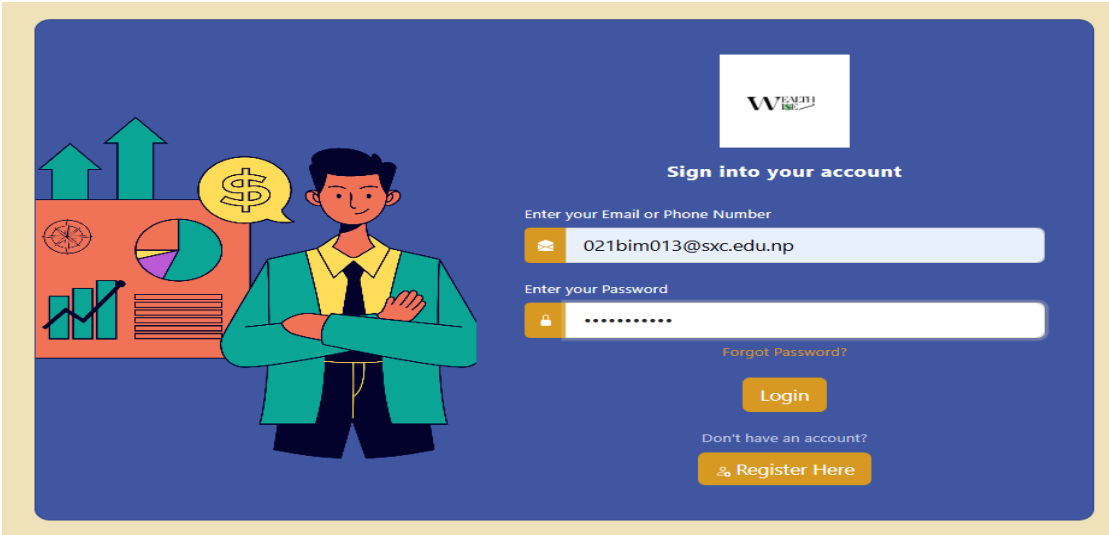
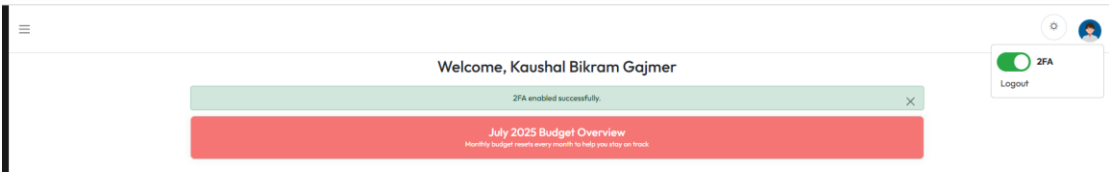


Figure 2.22: Login Without OTP



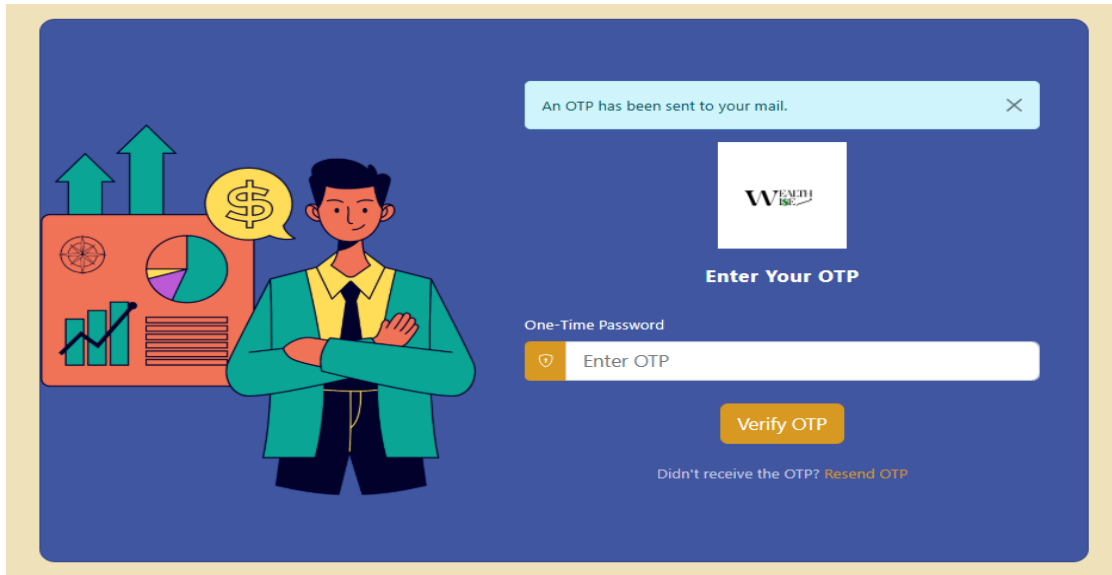


Figure 2.23: Login With OTP

Conclusion: The 2FA toggle correctly integrates with the login flow, enhancing user experience by allowing OTP to be optional.

2.3.6.3 System Testing

System testing involves testing the fully integrated software as a whole to verify that all modules work together to meet the project's requirements. Performed after integration testing, it evaluates end to end functionality, user interactions, and system performance to ensure the application delivers the expected results.

In WealthWise, system testing focused on performance, stress, and security.

Table 2.17: System Testing Test Cases

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Test Status
ST-001	Email Notification for Budget Threshold	1. Set Wants budget 2. Add expenses 3. Check email notification	Wants Budget Limit: 500 Wants Expense: 450	Email should be sent when one crosses 80% of the allocated budget limit for "Needs" or "Wants".	Email is sent	Pass
ST-002	Performance:	1. Open dashboard with 10	10 transactions	Load within 2s	Loaded in 1.25s	Pass

	Dashboard Load	transactions 2. Use a stopwatch to measure load time from click to full display				
ST-003	Stress: Concurrent Users	1. Open multiple browser tabs (5 tabs) with different users 2. Simultaneously access dashboard and add an expense in each tab	5 users, Amount: Rs 1000 per tab	System handles requests without crashing	All tabs processed successfully	Pass
ST-004	Security: SQL Injection	1. Attempt registration with malicious input 2. Check system response	Username: Kaushal' OR '1'='1 Email: test@example.com Phone: +9779764233148 Address: Hattiban Password: Password@123 Confirm Password: Password@123	Blocks unauthorized registration or rejects invalid input	Input rejected due to invalid characters	Pass

Test Case 1: Email Notification for Budget Threshold.

Expected Output: Test is successful if email is sent when threshold is reached.

Evidence:



Figure 2.24: Email Notification

Conclusion: Email notifications are sent correctly when budget threshold is reached.

Test Case 4: Security: SQL Injection

Expected Output: Test is successful if unauthorized registration is blocked or invalid input is rejected.

Evidence:



Figure 2.25: SQL Injection Test

Conclusion: System rejects invalid input containing malicious characters, ensuring security.

2.3.6.4 UAT

UAT, also called application testing or end-user testing, is a phase of software development in which the software is tested in the real world by its intended audience (Gillis,2022).

UAT was conducted that involved 5 end-users, testing WealthWise on a local setup. The evaluation process began with each user registering a new account using their details, ensuring they could navigate the registration form and successfully create an account. After logging in, users were guided through core functionalities that were adding income and expenses, categorizing expenses (Needs, Wants, Other), viewing budget allocations based on the 50/30/20 rule, and accessing the dashboard to see visualizations of their spending. The mail automation feature was demonstrated by adding expenses until spending crossed 80% of the budget limit for Wants triggering a warning email that users received in their inboxes. Users also interacted with the chatbot, asking questions like “How to save money?” to assess its advice. Feedback was highly positive with users appreciating the project.

Table 2.18: UAT Test Cases

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Test Status
UAT-001	Add Expense and Check Budget	1. Add expense 2. Verify budget update	Amount: 1000 Category: Needs	Budget should update correctly	Budget updated correctly	Pass
UAT-002	Chatbot Usability	1. Ask chatbot for budgeting advice	Query: How to save?	The chatbot should provide clear, concise advice	The chatbot provided clear, concise advice	Pass

2.3.7 Deployment

The deployment process is tailored to support its development, testing, and initial user trials, with a focus on a cost-effective and flexible hosting approach. The following details outline the current hosting strategy and infrastructure utilized to ensure the application's functionality and performance.

i) Hosting Approach:

WealthWise is primarily hosted locally on a laptop (Intel Core i5-1350H, 16GB RAM, 512GB SSD) for development and testing. The Flask application runs on its built-in development server, accessible via `http://127.0.0.1:5000` on the local machine or within the same network. XAMPP v3.3.0 provides MySQL 8.0 (MariaDB) as the database backend, integrated with Flask via the `flask_mysqldb` extension. PHPMyAdmin, bundled with XAMPP, facilitates database management. Mailpit is configured locally to test email functionalities, such as welcome and OTP emails, without external delivery. This local setup minimizes costs and supports rapid development and debugging.

Additionally, a secondary deployment was successfully implemented on Render, a cloud hosting platform, to explore scalability. The Render version required code modifications, including switching from MySQL to PostgreSQL (Render's managed database) and adapting email handling to comply with Render's environment. Despite the same frontend UI and functionalities, backend changes were necessary to support PostgreSQL connectivity via `psycopg2` and environment variable management for database credentials. This Render deployment serves as a proof-of-concept for cloud hosting but is not reflected in the final project documentation, which focuses on the local setup due to its completion prior to the Render experiment.

ii) Deployment Platform:

The primary deployment platform for WealthWise is On-Premises, using the laptop as the server. The Flask built-in development server hosts the application, with MySQL (via XAMPP) as the database backend. This setup avoids external cloud platforms like AWS or Heroku, aligning with budget constraints and the project's prototype scope, which does not require public internet-facing deployment. The On-Premises approach offers full control over the environment, simplifies debugging, and meets the project's requirements as documented in the final report.

As an exploratory step, WealthWise was deployed on Render to test cloud-based hosting. This required significant backend code changes, such as migrating from MySQL to PostgreSQL and updating database connection logic, while the frontend and all the functionalities remained unchanged. The Render deployment leverages Render's managed PostgreSQL database and environment variables for secure configuration. However, due to the finalized project documentation focusing on the local setup and the late-stage nature of the Render deployment, the On-Premises platform remains the documented standard.

CHAPTER III: CONCLUSION AND RECOMMENDATION

3.1 Summary

WealthWise is a financial management web application that effectively assists users in managing personal finances, particularly low income or pocket money, using the 50/30/20 budgeting method (50% needs, 30% wants, 20% savings). Built with Flask, MySQL, and Bootstrap, it integrates Chart.js for real-time visualizations, Pandas for data manipulation, Matplotlib and Seaborn for offline charts, Bcrypt for secure password hashing, and OllamaLLM for an AI chatbot. Features include secure OTP-based authentication (email/phone), monthly transaction tracking, automated email reminders/notifications via Mailpit, budget planning, interactive dashboards, and offline report generation (Excel and PNG), simplifying financial management with actionable insights.

3.1.1 Findings

The Finding section presents an overview of the key observations and insights gathered during the development and evaluation of WealthWise, highlighting the performance and challenges of the tools and technologies utilized. The findings are summarized in Table 3.1, which details the categories, specific tools/technologies, and their respective strengths and areas for improvement.

Table 3.1: Findings Table

Tool/Technology	Category	Findings
Bootstrap	CSS Framework	-Accelerated responsive UI design -Customization needed extra CSS for branding.
Chart.js	JavaScript Library	Enabled dynamic visualizations.
CSS	Style Sheet Language	-Enhanced UI control -Required careful observation for responsiveness.
Draw.io	Diagramming Tool	-Simplified DFD/ERD creation. -Diagram exports needed manual clarity adjustments.
GitHub	Version Control	-Streamlined collaboration. -Consistent commit messages needed for clear history.
VS Code	IDE / Code Editor	-Boosted productivity with extensions.

JavaScript	Scripting Language	-Powered dynamic chatbot/charts. -Async requests needed robust error handling.
Mailpit	Email Testing Tool	-Easy email testing. -Required thorough checks for consistent formatting.
MySQL	RDBMS	-Reliable storage via XAMPP -Schema changes needed careful migration management.
Ollama with Llama3	AI Model Framework	-Enhanced chatbot with advice. -Resource-heavy on i5-1350H, needed optimization.
Pandas, Matplotlib, Seaborn	Data Analysis/Visualization	-Excelled in report generation; -Large datasets required memory management.
Python with Flask	Backend Framework	-Enabled rapid server-side logic -Session/error handling needed careful debugging.
Render	Cloud Hosting Platform	-PostgreSQL migration was challenging.

3.1.2 Limitations of the Current System

Despite its functionality, WealthWise has limitations. It lacks mobile application support, restricting accessibility for users on the go. Real-time notifications for budget limits are absent, which could help users avoid overspending while logged in. The application currently supports only English language, limiting its reach in diverse regions like Nepal. Advanced accessibility features, such as voice input, are not implemented, potentially excluding users with disabilities. Additionally, Google Login is not available currently, requiring users to depend upon email and phone-based registration, which may reduce convenience for some users.

3.1.3 Future Enhancements

To address these gaps, future enhancements include:

- i. Developing a mobile application to improve accessibility.
- ii. Implementing real-time notifications to alert users when they approach or exceed budget limits.
- iii. Adding multi-language support (e.g., Nepali) for broader inclusivity in Nepal.
- iv. Introducing voice input capabilities to enhance accessibility for users with disabilities.
- v. Integrating Google Login to provide an additional authentication option, enhancing user convenience and seamless access.

3.1.4 Lessons Learned

The project provided significant learning opportunities in both technical and non-technical areas. Technically, skills were gained in Flask-based web development, MySQL database design, RESTful API implementation, and AI integration with OllamaLLM, alongside mastering data visualization with Chart.js, Matplotlib, and Seaborn. Non-technically, the development process improved problem-solving, time management, and iterative testing skills, particularly during challenges like session management, chatbot setup, and email notification configuration.

3.2 Conclusion

WealthWise contributes to the field of Computer Science and Information Management by demonstrating the practical application of web technologies, database management, automation and AI in solving real-world financial management problems. The integration of the 50/30/20 budgeting method with modern tools like Flask, MySQL, and OllamaLLM showcases how technology can simplify complex financial decision-making for individuals. This project lays a foundation for future research and development in personalized financial tools, highlighting the importance of user-centered design and secure, scalable systems in financial technology.

3.3 Recommendations

To further improve WealthWise, the following enhancements are recommended:

- i. Introducing a feature to import budget summaries from the user and providing users with advices based on their financial records.
- ii. Developing a mobile application for this project
- iii. Introducing real time notifications to alert users when they login, approach or exceed their budget limits for needs, wants, or savings.
- iv. Enhancing the system with voice input capabilities and more enhanced accessibility
- v. Introducing multi-language support to make the application accessible in languages like Nepali, catering to a broader audience in Nepal and improving user inclusivity.
- vi. Integrating Google Login to offer an additional authentication method, improving user convenience.

These improvements could be prioritized based on user feedback, with initial focus on google login, mobile integration and notifications, followed by language support and advanced features like voice input.